



Submitted in part fulfilment for the degree of BSc.

Deep Learning for Underwater Object Detection with a Focus on Fish Recognition and Classification

Mohamad Haikal Ubaidilah Bin Mohammad Nazli

02 May 2025

Supervisor: Dr. Pengcheng Liu

To all students everywhere

Acknowledgements

I would like to firstly thank my supervisors, Dr. Pencheng Liu and Dr. Rob Alexander for their continued support throughout the whole project. Without their guidance and mentorship, I would not have been able to successfully complete this project.

I would also like to specifically express my gratitude towards my parents and my siblings for their everlasting patience and care for me. Their perseverance and constant encouragement has made the person I am today.

Finally, I would like to show my appreciation to my friends that I made since my first year. The support and company I have received from everyone means the world to me more than you will ever know.

Contents

Acronyms	viii
Statement of Ethics	ix
Executive Summary	xi
1 Introduction	1
1.1 Context	1
1.2 Motivations	1
1.3 Objectives	2
1.4 Project Structure	2
2 Literature Review	3
2.1 Challenges in Underwater Environments	3
2.2 Traditional Methods and the Emergence of Deep Learning	5
2.3 Discussion on Image Degradation and Habitat Variability	7
2.4 Importance of Datasets and Emerging Trends	8
2.5 Promising Model Architectures	11
2.5.1 The YOLO Family: Real-Time Object Detection	11
2.5.2 Faster R-CNN: High Accuracy but Computationally Expensive	12
2.5.3 Swin Transformers: A Shift Towards Vision Transformers (ViTs)	13
2.5.4 EfficientNet: Balancing Accuracy and Efficiency	14
2.5.5 Hybrid Models: Combining CNNs and Transformers for Better Results	15
2.5.6 Choosing the Right Model: Trade-Offs and Application-Specific Considerations	16
2.5.7 Future Trends in Underwater Object Detection Architectures	17
2.6 Future Directions	17
2.6.1 Advanced Image Enhancement Techniques	18
2.6.2 Evolution of Model Architectures	18
2.6.3 Dataset Expansion and Cross-Domain Learning	19
2.7 Conclusion	20

Contents

3	Methodology	21
3.1	Data Collection and Preprocessing	21
3.1.1	Dataset Consideration and Selection	21
3.1.2	Dataset Preprocessing	25
3.2	Model Selection and Implementation	28
3.2.1	YOLOv8 Model	29
3.2.2	EfficientNet Model	38
3.2.3	Swin Transformer Model	47
3.3	Conclusion	56
4	Result and Evaluation	57
4.1	Evaluation of Model Performance	57
4.1.1	YOLOv8 Model	58
4.1.2	EfficientNet Model	65
4.1.3	Swin Transformer Model	70
4.2	Comparative Analysis of Model Performance	75
5	Conclusion and Future Work	79
5.1	Summary of Key Findings	79
5.2	Discussion of Limitations	80
5.3	Recommendations for Future Work	80
5.4	Final Closing Remarks	81
A	Dataset Access	83
B	Figures	84
C	Tables	94

List of Figures

2.1	YOLO Architecture Overview	12
2.2	Faster R-CNN Architecture Overview	13
2.3	Vision Transformer Architecture Overview	14
2.4	EfficientNet Architecture Overview	15
2.5	FBDPN Architecture Overview	16
3.1	YOLOv8 output head architecture for multi-scale detection .	30
3.2	Summary of EfficientNet-B0 Architecture	40
3.3	(a) The architecture of a Swin Transformer (Swin-T); (b) two successive Swin Transformer Blocks	48
4.1	YOLOv8 Normalized Confusion Matrix	61
4.2	YOLOv8 F1-Confidence Curve	62
4.3	YOLOv8 Precision-Recall Curve	63
4.4	EfficientNet Training and Validation Accuracy over Epochs .	66
4.5	EfficientNet Training and Validation Loss over Epochs . . .	66
4.6	EfficientNet Normalized Confusion Matrix	68
4.7	Swin Transformer Training and Validation Accuracy over Epochs	71
4.8	Swin Transformer Training and Validation Loss over Epochs	72
4.9	Swin Transformer Normalized Confusion Matrix	73
B.1	Image sample of the train set with bounding box derived from the segmentation mask	85
B.2	Screenshot of the terminal after the dataset preprocess . .	85
B.3	YOLOv8 Confusion Matrix	86
B.4	YOLOv8 Precision-Confidence Curve	86
B.5	YOLOv8 Recall-Confidence Curve	87
B.6	YOLOv8 Learning Rates over Epochs	87
B.7	YOLOv8 Trainig Losses over Epochs	88
B.8	YOLOv8 Validation Losses over Epochs	88
B.9	YOLOv8 Precision and Recall over Epochs	89
B.10	YOLOv8 Mean Average Precision (mAP) over Epochs . . .	89
B.11	Sample of YOLOv8 Prediction on Test Set	90
B.12	Sample of Ground Truth Annotation on Test Set	90
B.13	Sample of EfficientNet Predictions on Test Set (Grid 1) . . .	91
B.14	Sample of EfficientNet Predictions on Test Set (Grid 2) . . .	92
B.15	Learning Rate Evolution over Epochs (Cosine Annealing) .	92

List of Figures

B.16 Sample of Swin Transformer Predictions on Test Set (Grid 1)	93
B.17 Sample of Swin Transformer Predictions on Test Set (Grid 2)	93

List of Tables

3.1	YOLOv8 Training Configuration	37
3.2	EfficientNet Training Configuration	46
3.3	Swin Transformer Training Configuration	55
4.1	Summary of YOLOv8 Model Performance on Fish4Knowledge Dataset	64
4.2	Summary of EfficientNet Model Performance on Fish4Knowledge Dataset	69
4.3	Summary of Swin Transformer Model Performance on Fish4Knowledge Dataset	74
C.1	Choosing the Right Model: Trade-Offs and Application-Specific Considerations	95
C.2	Summary of Datasets Considered for Fish Detection and Classification	96
C.3	Swin Transformer Early Training Results Before Improvements	96
C.4	Swin Transformer Training Results After Improvements . . .	96
C.5	Comparative Performance Metrics Across Models Trained on Fish4Knowledge Dataset	97

Acronym

UOD Underwater Object Detection

AI Artificial Intelligence

ML Machine Learning

CNN Convolutional Neural Network

YOLO You Only Look Once

R-CNN Region-based Convolutional Neural Network

FRCNN Faster Region-based Convolutional Neural Network

ViT Vision Transformer

SSD Single Shot MultiBox Detector

GAN Generative Adversarial Network

mAP mean Average Precision

IoU Intersection over Union

CEH-YOLO Composite Enhanced High-order YOLO

HDA High-order Deformable Attention

AUV Autonomous Underwater Vehicles

ROV Remotely Operated Vehicles

ESPPF Enhanced Spatial Pyramid Pooling Fast

UDA Unsupervised Domain Adaptation

Statement of Ethics

This project aims to develop and evaluate deep learning models for underwater object detection, specifically focusing on fish species in challenging aquatic environments. The study utilizes various state-of-the-art object detection models, including YOLOv8, EfficientNet, and Swin Transformer, to process underwater imagery under conditions of low visibility, variable lighting, and high environmental noise. The primary objective is to improve real-time detection accuracy and robustness across diverse underwater settings.

Use of External Datasets

The dataset used in this project is the publicly available *Fish4Knowledge* dataset, which contains annotated underwater images of various fish species. The dataset is hosted by the University of Edinburgh and is intended for academic research and non-commercial purposes. It can be accessed by clicking [here](#).

Upon review of the dataset's official webpage and supporting documentation, it is evident that:

- The dataset does **not** contain any human data or involve human participants.
- The dataset is **freely available** for academic use and does **not require explicit permission** from the data provider.
- The dataset was created for scientific purposes and has **no dissemination or licensing restrictions** that would limit its use in student research.

Ethical Risks and Mitigation

This research does not pose any ethical risks related to:

- Human or animal experimentation,
- Dual-use technologies or military applications,
- Involvement of vulnerable populations or data protection concerns.

The project does not involve collaboration with controversial industries, nor is it funded by any organization that could present a conflict of interest.

Based on a completed ethical checklist and declaration, all applicable categories of potential ethical concern were answered as “No”, indicating that **no formal ethical approval is required**.

Conclusion

All ethical considerations relevant to the dataset and methodology have been carefully reviewed. The project complies fully with the ethical standards and guidelines of the University. No further ethical approval is required.

Executive Summary

In recent years, there has been plethora of research on the utilization of object detection in real-world scenarios. One particular area where object detection clearly thrives in is underwater environments especially for marine biology research, aquaculture management, and autonomous underwater vehicle navigation (AUV). Despite that, underwater object detection still remains a challenging area of study due to the fact that aquatic environments have many variables and factors that need to be taken into account such as low visibility, uneven lighting, turbidity, and varying levels of image distortion, which affect the reliability and accuracy of object detection systems in these situations. There is an increasing need for robust detection systems with the capability of not only detecting but also identifying fish species for deployment in biomass estimation, biodiversity tracking, and other practical applications. With that being said, traditional object detection models often underperform in aquatic contexts due to environmental constraints as well as the lack of high-quality, diverse datasets despite the emerging of new model architectures. Accurate automated detection of marine life significantly reduces manual labour, minimizes human error, and offers substantial ecological and commercial benefits. Enhancing model accuracy and robustness could profoundly impact ecological monitoring and aquaculture operations by enabling real-time automated analyses.

This project aims to develop and test reliable deep learning models tailored specifically for detecting objects underwater, with a primary focus on accurately detecting and identifying different fish species, even in challenging underwater conditions. The goal is to improve real-time detection, which is crucial for tasks such as managing sustainable aquaculture, protecting marine biodiversity, and supporting autonomous underwater exploration.

Three state-of-the-art deep learning architectures —YOLOv8, EfficientNet, and Swin Transformer, all of which were based on their smallest variant—were evaluated using the publicly available Fish4Knowledge dataset, which provides over 27000 annotated underwater images and segmentation masks of 23 fish species. Data preparation involved converting segmentation masks into bounding boxes, formatting annotations compatible with YOLOv8 standards, and ensuring the dataset was split into training (70%), validation (20%), and testing (10%) subsets. A systematic preprocessing workflow was developed to automate annotation and validation processes,

ensuring accurate and reproducible results. The preprocessed datasets can be referred to in the Appendix A

In regards to the training procedures, it involved transfer learning methods, leveraging pretrained weights from large-scale datasets such as ImageNet and COCO to improve convergence and performance with the given time and resource constraints, rendering training from scratch unfeasible. Model evaluations and comparison utilized quantitative metrics including precision, recall, F1-score, mean Average Precision (mAP), and accuracy, complemented by qualitative assessments via confusion matrices and visualizations of predictions and the actual ground truth.

The comparative performance analysis on the three deep learning model revealed that Swin Transformer achieved the highest overall classification performance, attaining a final test accuracy of 99.70%, a precision of 98.70%, a recall of 97.4%, and an F1-score of 97.60%, although it was the largest model out of all which stood at $\sim 323\text{MB}$. As for EfficientNet, it offered a strong balance between performance and computational efficiency, achieving test accuracy of 99.10%, a precision of 98.50%, a recall of 94.8%, and an F1-score of 96.20%, while having significantly smaller model footprint at $\sim 50\text{MB}$ relative to Swin Transformer. For the third and last model, despite it being the smallest model size at just $\sim 12\text{MB}$ compared to the other models, it achieved strong detection performance, with mAP@0.5 reaching 0.981, a precision of 97.3%, a recall value of 97.8%, and an F1-score of also 97.0%.

The Swin Transformer, leveraging hierarchical Vision Transformer architecture, demonstrated superior performance, robust generalization, and rapid convergence, making it the most effective model tested for underwater species classification.

This project confirms the viability and effectiveness of deep learning techniques, particularly transformer-based models, for underwater object detection. Despite high performance, minor confusion among similar species indicates scope for further enhancement, possibly through expanded datasets and more sophisticated image preprocessing techniques. As for the future work recommendations, this include exploring hybrid architectures combining CNNs and transformers, advanced image enhancement methods to improve underwater visibility, and expanding training datasets to include a wider variety of aquatic life and environmental conditions. These steps would further refine the models' accuracy and generalizability for practical deployment in marine ecosystems monitoring, aquaculture management, and other applications.

No legal, social, ethical, professional, and commercial issues have been identified during this project.

1 Introduction

1.1 Context

The rapid development of deep learning has significantly advanced the field of object detection, enabling accurate and real-time recognition of diverse objects in terrestrial environments. However, underwater object detection remains a challenging subdomain due to the harsh and complex conditions of aquatic environments. Factors such as low visibility, uneven lighting, turbidity, and image distortion affect the reliability of object detection systems in these scenarios. In the field of aquaculture and marine monitoring, there is an increasing need for robust detection systems capable of identifying fish species for applications ranging from biomass estimation to biodiversity tracking. However, traditional object detection models often underperform in underwater contexts due to environmental constraints and the lack of high-quality, diverse datasets.

1.2 Motivations

The ability to detect and classify fish accurately in underwater imagery has direct implications for sustainable aquaculture, ecosystem management, and autonomous underwater exploration. Manual analysis of underwater footage is labor-intensive, time-consuming, and subject to human error. By enhancing the capabilities of machine vision in these settings, aquaculture operations can automate monitoring processes, minimize operational costs, and support conservation efforts. Moreover, the research contributes to the broader goal of enabling reliable AI performance in adverse or out-of-distribution environments. Given the unique visual challenges underwater, this project seeks to develop a specialized approach tailored to these conditions, improving upon the shortcomings of existing models.

1.3 Objectives

The primary objective of this dissertation is to develop a deep learning-based detection model capable of identifying fish in underwater images and possibly video streams with high accuracy. Specific objectives include:

- Conducting a thorough literature review of current underwater object detection techniques and identifying existing gaps.
- Preprocessing and augmenting relevant underwater datasets (e.g., Fish4Knowledge) to improve model robustness.
- Fine-tuning and evaluating multiple pre-trained deep learning models (e.g., YOLO, EfficientNet, Swin Transformer) for fish detection tasks.
- Comparing the models using metrics such as accuracy, precision, recall, F1-score, and Intersection over Union (IoU) under varying underwater conditions.

The success criterion is achieving at least 85% accuracy, precision, and recall, as well as high IoU scores, across challenging underwater environment.

1.4 Project Structure

This dissertation is structured as follows:

- **Chapter 2: Literature Review** provides a comprehensive review of the literature, focusing on recent advances in underwater object detection and the challenges posed by the aquatic domain.
- **Chapter 3: Methodology and Design** outlines the methodology used in the project, including dataset preparation, model selection, and training procedures.
- **Chapter 4: Implementation** presents the implementation of different models and preprocessing techniques.
- **Chapter 5: Result and Evaluation** evaluates the models based on established metrics and discusses the results.
- **Chapter 6: Conclusion** concludes the dissertation by summarising findings, discussing limitations, and suggesting directions for future work.

2 Literature Review

Object detection in underwater environments presents unique challenges due to factors such as light attenuation, turbidity, optical distortion, complex background, and the need to identify small objects. These environmental conditions degrade image quality, reducing the effectiveness of traditional object detection methods. The emergence of deep learning has revolutionized this field by enabling models to handle low-visibility, high-noise conditions with greater accuracy. This literature review explores the recent advancements in deep learning for underwater object detection, focusing specifically on the detection and classification of fish in challenging aquatic environments. By examining recent methodologies, datasets, and model architectures, this review aims to highlight key challenges, emerging trends, and future directions in this rapidly evolving domain.

Despite considerable progress, a clear research gap remains in the evaluating and comparing the generalization capabilities of state-of-the-art deep learning models under varying underwater conditions, particularly with datasets such as Fish4Knowledge. Current studies either focus on single-model performance or lack comprehensive comparisons involving newer architectures like YOLOv8. In this study, the gap will be addressed by developing and training deep learning models and evaluating their adaptability to degraded underwater scenarios. Additionally, this work will be extended by benchmarking other advanced models—namely EfficientNet and Swin Transformer—under the same conditions. These efforts aim to inform the design of a robots detection framework capable of adapting to the inherent complexities of underwater environments.

2.1 Challenges in Underwater Environments

Underwater object detection is significantly hindered by a range of environmental factors that degrade image quality and complicate feature extraction. These challenges primarily stem from light attenuation, turbidity, optical distortion, and background variability, all of which must be addressed for effective deep learning-based detection models.

First and foremost, unlike terrestrial environments, where light travels un-

impeded, underwater settings are characterized by selective wavelength absorption, leading to significant colour distortions. Red light is absorbed within the first few meters, followed by orange and yellow, leaving predominantly blue-green tones in deeper waters. This results in unnatural colouration of objects, making it difficult for deep learning models to correctly identify object features based on traditional colour segmentation techniques [1]. Moreover, light scattering due to water molecules and suspended particles further reduces visibility, creating a hazy appearance in images [2].

Apart from that, underwater imaging systems experience refractive distortions due to differences in the refractive index between air, water, and glass (camera lenses or robotic vision systems). This leads to geometric distortions, where objects appear displaced or stretched depending on the angle of capture. Additionally, the presence of currents and moving marine life contributes to motion blur, making it difficult for object detection algorithms to correctly delineate object boundaries, especially in low-light conditions [1].

Moreover, underwater environments often have highly dynamic and cluttered backgrounds, including rocks, corals, seaweed, and artificial structures (e.g., fish farms). This variability can cause deep learning models trained on clear datasets to fail when exposed to new environments. Furthermore, many marine species exhibit natural camouflage, using colour adaptation or transparent bodies to blend into their surroundings, reducing the effectiveness of standard object detection models [2].

Given these challenges, conventional image preprocessing techniques such as histogram equalization and white balancing are insufficient for correcting underwater images. Recent advancements leverage deep learning-based enhancement techniques, including CEH-YOLO's Enhanced Spatial Pyramid Pooling (ESPPF) module, which improves feature extraction for small, occluded, and low-contrast objects. Other models integrate dehazing, contrast enhancement, and generative adversarial networks (GANs) to restore lost image details before object detection [1] [3].

Addressing these challenges requires domain-specific deep learning models that incorporate adaptive preprocessing, robust feature extraction, and multimodal sensing techniques (e.g., fusing optical and sonar data). Unlike traditional methods that rely on handcrafted features, modern deep learning models such as YOLO, Faster R-CNN, and Swin Transformers, automatically learn to detect and classify objects in dynamic underwater environments [4] [3].

For instance, YOLO-based models are designed for real-time detection, making them effective for marine applications where rapid identification is

crucial. Faster R-CNN, though computationally heavier, excels in precise object localization, helping to detect small and occluded marine species. Meanwhile, Swin Transformers leverage self-attention mechanisms, allowing them to capture long-range dependencies and improve detection in complex marine environments with cluttered backgrounds [4].

By leveraging these advancements, deep learning-driven approaches have demonstrated superior adaptability and accuracy compared to traditional techniques. The next section discusses the shift from traditional object detection methods to deep learning-driven approaches that have proven more effective in handling underwater imaging complexities.

2.2 Traditional Methods and the Emergence of Deep Learning

Traditional computer vision techniques for underwater object detection have relied heavily on handcrafted feature extraction methods, which involve manually designing algorithms to detect objects based on shape, colour, and texture. To improve object detection in challenging underwater environments, researchers have explored shape-based level set frameworks. These frameworks help differentiate objects from the background by gradually refining the edges of detected objects, making them more distinguishable even when partially hidden or overlapping with other elements [5] [2].

Another approach involves deformable multi-core technology, which helps compensate for camera movement and environmental changes. This technique allows object detection models to adjust dynamically when the camera or marine life is moving, reducing errors caused by motion blur or sudden changes in water conditions. While these methods offer some improvements, they are highly dependent on environmental conditions and often struggle in highly turbid waters, extreme lighting conditions, or complex marine settings where objects blend into their surroundings [2] [6].

Before the rise of deep learning, many underwater object detection systems used a simpler technique called threshold segmentation. This method works by comparing the brightness of each part of the image and separating it into "object" or "background" based on a set cutoff value. It is like turning a photo into a black-and-white sketch based on how light or dark different parts are. However, underwater images often have uneven lighting and unclear boundaries, which makes it hard for a single cutoff point to work well everywhere in the image. To improve this, a more flexible version was introduced by Zhu and team. Instead of using just one threshold for the

whole image, their method adjusts the threshold for each pixel based on the brightness of nearby areas. This makes the image edges stand out more clearly, even in messy or unclear underwater scenes. Although modern AI models are more powerful today, this type of thresholding is still useful for quick and simple image processing, especially when computing resources are limited [7].

Besides traditional camera-based methods, a newer and more effective approach for underwater object detection is combining both optical (visual) and sonar (sound-based) images. Underwater cameras can capture colourful, detailed pictures, but their visibility is often limited by murky water and poor lighting. On the other hand, sonar can "see" through cloudy water using sound waves, but the images it creates are usually blurry and lack colour. By fusing these two types of images, researchers can get the best of both worlds—clearer shapes from sonar and colour details from optical cameras. For example, a study by Kim and colleagues introduced a system that blends these two image types into one, making underwater scenes much easier to understand. Their fusion method significantly improved image clarity and helped distinguish objects more accurately. This kind of technology is especially useful for underwater robots and remote-controlled vehicles that need to operate safely and effectively in deep or unclear waters [3].

The emergence of deep learning has significantly advanced underwater object detection by enabling models to learn hierarchical feature representations, capturing both low-level details (edges, textures) and high-level semantic information (object shapes, contextual cues) [2]. Unlike traditional methods that rely on manually engineered features, deep learning models automatically extract relevant features from data, making them more adaptable to variable underwater environments. Early implementations of Convolutional Neural Networks (CNNs) in underwater fish detection demonstrated superior accuracy but were computationally expensive and suffered from slow inference speeds, limiting their applicability in real-time scenarios. This limitation spurred the development of more efficient architectures, including Generative Adversarial Networks (GANs) for image enhancement, Single Shot Detector (SSD) for fast object localization, and You Only Look Once (YOLO) for real-time detection. These architectures have been further adapted to incorporate underwater-specific image features, such as colour correction, dehazing, and noise reduction, improving detection accuracy in challenging aquatic conditions [2] [4].

By shifting from handcrafted feature extraction to end-to-end learning models, deep learning has established itself as the dominant paradigm for underwater object detection. The following sections explore how modern deep learning architectures address image degradation and habitat variability, further enhancing detection robustness in complex marine environments.

2.3 Discussion on Image Degradation and Habitat Variability

One of the most significant challenges in underwater object detection is the degradation of image quality due to environmental factors such as light attenuation, turbidity, and optical distortions. To overcome this, researchers have focused on integrating image enhancement techniques with deep learning models, aiming to improve the quality of underwater images before they are processed by object detection algorithms [5] [1]. This preprocessing step is crucial, as it allows models to extract more meaningful features from degraded images, thereby improving detection accuracy in low-visibility conditions.

Various enhancement techniques have been explored, including contrast adjustment, histogram equalization, Retinex-based methods, and generative adversarial networks (GANs). Studies indicate that deep learning-based physical prior knowledge models, which leverage information about light propagation in water, can significantly improve the perception effect of object detection. However, a fundamental challenge remains: image enhancement and object detection have different optimization goals. While enhanced images may appear more visually appealing, they can introduce artificial distortions, loss of structural details, or over-smoothing, which may negatively impact feature extraction for object detection models, especially in cases involving small or camouflaged marine species [5] [2] [8].

This challenge highlights the trade-off between visual enhancement and feature preservation. While an image may appear cleaner to the human eye, it may lose edge sharpness or fine-grained textures necessary for accurate detection. To address this, researchers have begun exploring joint optimization strategies that integrate enhancement and detection into a unified framework. For example, Zheng et al. [2] proposed a novel model combining a Prompt-Based Degradation Feature Learning Module (DFLM) with a two-stage object detection pipeline. The DFLM operates in the Fourier domain to capture degradation-aware image features, which are then passed directly into the object detection network. This joint setup allows the model to learn from both high-quality and degraded images, optimizing for both visual fidelity and detection accuracy during training. Unlike traditional approaches that treat enhancement as a separate preprocessing step, this end-to-end design enables more effective feature retention in complex underwater conditions [2] [8].

In addition, convolutional neural networks (CNNs) which serve as the backbone for many object detection models, are not without limitations. Common issues include vanishing gradients in deeper layers, susceptibility

to overfitting on limited datasets, and even sensitivity to adversarial noise, which can be problematic in environments with high variability such as murky or cluttered underwater scenes. These limitations necessitate further architectural improvements or replacements, such as attention-based mechanisms or hybrid CNN-transformer models, that can better handle such conditions [1] [6] [4].

Beyond image degradation, underwater habitat variability presents another major obstacle. Fish inhabit diverse ecological environments, including coral reefs, open water, estuaries, and deep-sea zones, each with unique visual characteristics. A model trained on images from one habitat may struggle to generalize to another due to differences in background clutter, foreground camouflage, and structural complexity. For instance, some species blend seamlessly with their surroundings, making feature extraction more challenging in highly textured environments like coral reefs. [2]

To address this issue, researchers employ two primary strategies. The first involves data augmentation techniques, where models are trained on synthetically modified images that simulate various underwater conditions, including changes in brightness, turbidity, and background composition. This improves a model's ability to generalize across different aquatic habitats [2]. The second strategy involves structural modifications to deep learning architectures, such as feature fusion networks, domain adaptation techniques, and attention mechanisms, which help models adapt to diverse underwater environments. For example, some models integrate multi-scale feature extraction to detect objects at different resolutions, enhancing their robustness to background variability and occlusion [9].

As underwater object detection continues to evolve, the integration of adaptive enhancement techniques with habitat-aware deep learning models will be essential in ensuring consistent performance across diverse underwater ecosystems. Future research will likely focus on end-to-end optimization, allowing image restoration and object detection to work in tandem, as well as cross-domain adaptation techniques to improve model transferability across different aquatic environments. Ultimately, striking the right balance between image enhancement quality and feature retention will be essential to ensure both visual fidelity and high detection accuracy.

2.4 Importance of Datasets and Emerging Trends

The selection of appropriate datasets plays a critical role in the development and evaluation of deep learning models for underwater object detection.

Unlike terrestrial datasets, underwater datasets must account for unique environmental challenges, such as light attenuation, turbidity, and varying habitat structures, which significantly impact model performance. Publicly available datasets on Kaggle, such as Fish-Pak and A Large Scale Fish Dataset, have primarily focused on marine environments, making them more suitable for deep-sea applications. However, these datasets still exhibit limitations in their adaptability to aquaculture settings, where controlled monitoring conditions differ from open-ocean environments. This underscores the need for specialized datasets tailored to specific marine species and conditions, ensuring that deep learning models are trained on images that accurately reflect the environments in which they will be deployed.

Addressing Dataset Limitations

One of the main challenges in underwater datasets is the quality and diversity of training images. Many datasets, such as A Large Scale Fish Dataset, Fish-Pak and others mentioned previously, are heavily biased toward single fish in clear-water, well-lit marine environments, with limited representation of turbid, low-contrast, or deeper water conditions. This bias can lead to deep learning models that overfit to ideal scenarios, reducing their effectiveness in real-world applications where underwater visibility and lighting vary drastically [6]. For instance, A Large Scale Fish Dataset and the Fish-Pak Dataset primarily consists of images of captured fishes which are outside of their habitat, hence, they have consistent lighting and minimal particulate matter. As a result, models trained solely on these datasets may fail to generalize to more challenging conditions such as murky estuaries, deep-sea habitats, or environments with strong backscatter and motion blur. To mitigate this issue, researchers are exploring dataset structuring techniques, such as dividing datasets based on degradation severity. By classifying images according to their visibility, contrast levels, and turbidity, models can be trained on progressively degraded datasets, improving their robustness in real-world applications. This tiered training approach allows models to gradually adapt to low-quality inputs, resulting in improved generalization across varying underwater conditions [2].

Additionally, the use of image quality assessment (IQA) metrics has gained attention as a method for curating datasets specifically optimized for object detection. IQA-based dataset construction involves analyzing sharpness, contrast, and structural integrity to filter images that are most conducive to successful object detection. This ensures that models are trained on datasets that strike a balance between visual realism and feature distinctiveness, enhancing their ability to detect objects in challenging underwater conditions [2].

Emerging Trends in Underwater Dataset Construction

Recent research has also investigated the impact of image enhancement on detection performance by comparing models trained on raw underwater images vs. preprocessed (enhanced) datasets. While enhancement techniques, such as GAN-based restoration and Retinex filtering, improve human-perceived image quality, they do not always translate to higher detection accuracy. Some studies suggest that overly enhanced images alter object textures and edge structures, leading to feature misalignment in detection models. Consequently, researchers are developing adaptive training pipelines where image enhancement is integrated as part of the model's feature extraction process, rather than a separate preprocessing step [9] [4].

Another emerging trend is the use of synthetic underwater datasets generated through simulation frameworks. These datasets allow researchers to model diverse environmental factors such as light attenuation, turbidity, colour distortion, and object occlusion, creating controlled training scenarios for object detection. While synthetic datasets can significantly improve training diversity and volume, research shows they are not sufficient on their own for achieving high generalization in real-world settings. This is because models trained exclusively on synthetic data often struggle to adapt to the complexity and unpredictability of actual underwater environments [9] [8].

To mitigate this, researchers advocate for combining synthetic data with real-world images during training. This hybrid approach leverages the strengths of both: synthetic data provides diverse degradations, while real data anchors the model in authentic visual patterns [2]. In the UFD2022 dataset study, for instance, the researchers created three types of training image sets, the original dataset with unmodified underwater images (Domain-O), a version enhanced using one type of image improvement algorithm (Domain-D), and another version enhanced using a different technique, including GAN-based methods (Domain-L). Rather than training models on just one version of the data, the study found that combining all three versions (Domain-O + Domain-L + Domain-D) during training helped the models perform more reliably. This mix allowed the models to learn from both naturally degraded images and visually improved ones, leading to better results when applied to new, unseen underwater environments [2] [9].

Furthermore, the generalizability of synthetic data is validated through cross-domain evaluations, where models trained on synthetic-enhanced domains are tested on real-world degraded images. These evaluations have shown that domain shifts still negatively impact accuracy, confirming the necessity of cross-domain fine-tuning or domain adaptation techniques for robust underwater object detection [2].

Future Directions in Underwater Dataset Research

Moving forward, the integration of multi-sensor datasets combining optical, sonar, and LiDAR-based imaging is expected to enhance underwater object detection models. By fusing data from different imaging modalities, researchers aim to overcome the limitations of optical cameras in murky waters [3]. Furthermore, semi-supervised and unsupervised learning approaches are gaining traction, as they allow models to learn from large-scale unannotated datasets, reducing the reliance on expensive manual labeling efforts [4].

As underwater deep learning continues to evolve, dataset advancements will play a pivotal role in shaping model reliability and real-world adaptability. The ongoing development of environment-specific, quality-aware, and multi-modal datasets will ensure that future deep learning models are better equipped to handle the complexities of underwater object detection [3].

2.5 Promising Model Architectures

The selection of an appropriate deep learning model is critical for effective underwater object detection, as different architectures offer trade-offs between accuracy, speed, and computational efficiency. While convolutional neural networks (CNNs) have traditionally been the backbone of object detection, advancements in transformer-based models and hybrid architectures are redefining the landscape. Several models, including YOLO, Faster R-CNN, SSD, and Swin Transformer, have demonstrated promise in addressing the unique challenges of underwater environments [1] [2] [6].

2.5.1 The YOLO Family: Real-Time Object Detection

One of the most widely used architectures for underwater detection is YOLO (You Only Look Once), known for its speed and accuracy in real-time applications. YOLO's single-stage detection pipeline allows for rapid inference, making it ideal for live tracking in underwater robotics and aquaculture monitoring [1] [6].

Recent iterations, such as YOLOv8, incorporate enhanced spatial pyramid pooling (SPP), anchor-free detection, and improved feature extraction, boosting detection accuracy for small, occluded, and low-contrast underwater objects [2]. However, YOLO's one-stage approach can sometimes lead

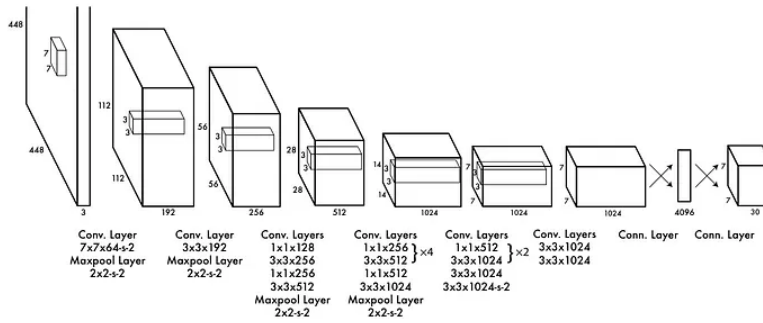


Figure 2.1: YOLO Architecture Overview

to lower localization accuracy compared to two-stage models, particularly in highly cluttered underwater environments [2].

A notable variant of YOLO, CEH-YOLO, integrates high-order deformable attention and an enhanced spatial pyramid pooling-fast (ESPPF) module, achieving impressive mean average precision (mAP) scores on benchmark datasets such as DUO and UTDAC2020. These improvements allow CEH-YOLO to excel in detecting small, camouflaged fish, making it a strong candidate for challenging underwater scenarios [2].

Figure 2.1 below shows the architecture of a typical YOLO model. It starts with a large input image, which is gradually reduced in size as it passes through several convolutional and pooling layers. These layers help the model detect key features like edges, shapes, and textures. Toward the end, fully connected layers are used to predict object classes and their locations within the image. This single-pass structure is what makes YOLO fast and efficient for real-time detection [10].

2.5.2 Faster R-CNN: High Accuracy but Computationally Expensive

Faster R-CNN is one of the most accurate object detection models and has been widely used in underwater research due to its strength in locating and classifying small or overlapping marine objects [1]. Unlike YOLO, which detects everything in a single pass, Faster R-CNN follows a two-step process. First, it uses a Region Proposal Network (RPN) to identify potential object areas in an image, then it classifies each proposed region using a Fast R-CNN module [11].

As shown in Figure 2.2, the architecture usually starts with a deep convolutional network (such as VGG16 or ResNet) to extract features from the input image. The RPN then scans these features and suggests boxes

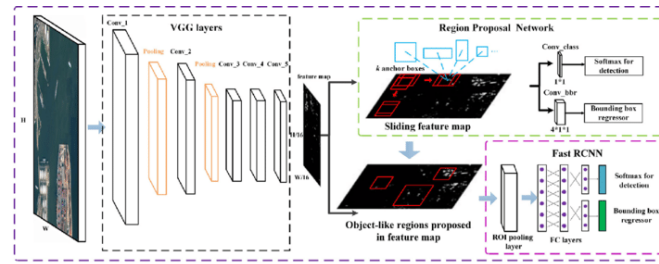


Figure 2.2: Faster R-CNN Architecture Overview

(called anchors) where objects might be. These boxes are passed to the second stage, which further refines them and assigns class labels (e.g., fish, background).

This two-step structure enables high precision and tighter bounding boxes, which is especially helpful when dealing with marine species that are small or partially hidden. However, this also means that Faster R-CNN is slower and more computationally heavy than single-shot models like YOLO or SSD, making it less suitable for real-time tasks in resource-limited environments such as underwater drones [1] [2] [11].

2.5.3 Swin Transformers: A Shift Towards Vision Transformers (ViTs)

Transformers, originally designed for natural language processing, are now being adapted for computer vision tasks including underwater object detection. The Vision Transformer (ViT) breaks an image into small patches (like tiny image tiles), flattens them into sequences (like words in a sentence), and processes them using a Transformer Encoder as seen in Figure 2.3. This lets the model focus on both the fine details of each patch and the broader context across the image, an ability that is especially valuable in underwater scenes where animals often blend into complex backgrounds.

Swin Transformer improves on this by introducing a hierarchical design and local self-attention within image windows, making it more efficient and scalable than the original ViT. It has shown strong results in handling challenges like background clutter and varying object sizes in underwater environments [12].

Although still relatively new to this domain, transformer-based models are becoming increasingly promising due to their better understanding of spatial relationships (important for detecting camouflaged fish), strong generalisation across different underwater environments, and ability to work well with large and complex datasets.

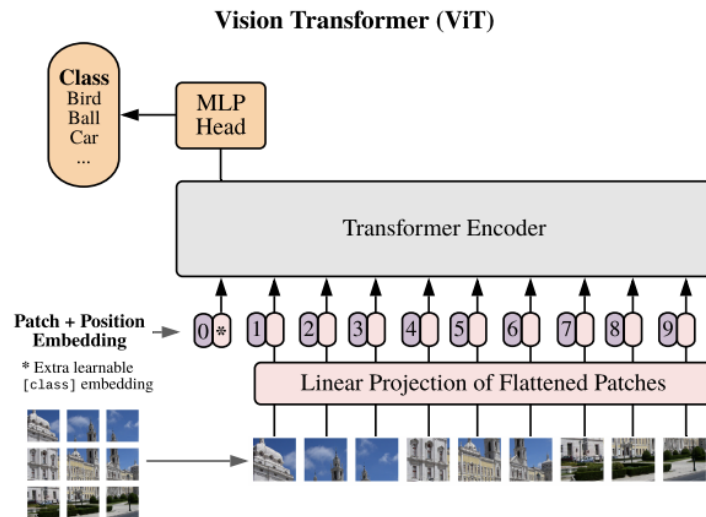


Figure 2.3: Vision Transformer Architecture Overview

However, they do have drawbacks, like being computationally heavy and requiring large amounts of labeled training data. These factors currently limit their real-time use in underwater drones or field equipment.

2.5.4 EfficientNet: Balancing Accuracy and Efficiency

EfficientNet stands out as a deep learning model that offers a great balance between high accuracy and low computational cost. Traditional models often become slower and bulkier as they grow in size, but EfficientNet solves this by the way it builds its structure using blocks called MBConv layers, which are optimized for performance, and how it balances depth (number of layers), width (layer size), and resolution (image size) using a strategy known as compound scaling. This approach helps the model learn detailed features without becoming too large or slow [13].

In underwater object detection, EfficientNet works especially well because it can extract fine details (like small fish) while also being efficient enough to run on systems with limited hardware, such as underwater drones or edge devices.

Figure 2.4 illustrates how an image passes through multiple EfficientNet blocks, each responsible for learning different features, from simple edges to complex object shapes. These blocks help the model recognize marine creatures even in murky or low-contrast waters.

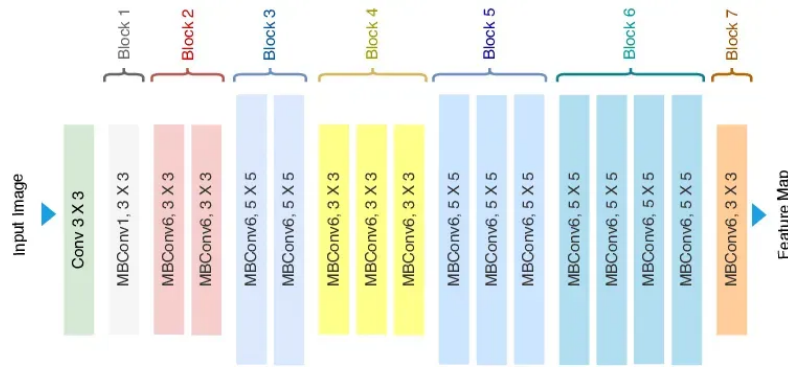


Figure 2.4: EfficientNet Architecture Overview

2.5.5 Hybrid Models: Combining CNNs and Transformers for Better Results

Recently, researchers have started combining two types of neural networks, Convolutional Neural Networks (CNNs) and Transformers, into what's known as hybrid models. CNNs are excellent at detecting small details, while Transformers are great at understanding the broader context of an image. When used together, these models can pick up both local details (like the outline of a fish) and long-range patterns (like fish blending into coral) [14].

A strong example of this is the FBDPN model (Feature Boosting and Differential Pyramid Network). This architecture uses a CNN-based backbone (such as ResNet50) to extract early visual features and a combination of modules (NSFBM and CSFDM) in the “neck” to refine and boost feature quality. The model then passes this enhanced information to a transformer-style head for final prediction. This multi-stage design allows the model to handle blurred, small, or camouflaged underwater objects much better than traditional CNNs alone.

Figure 2.5 which is taken from the original study [14] shows the FBDPN architecture, with a backbone for early feature extraction, a neck for feature enhancement, and a head for final object prediction. This design significantly boosts performance in challenging underwater scenes by combining detailed local patterns with a broader contextual understanding.

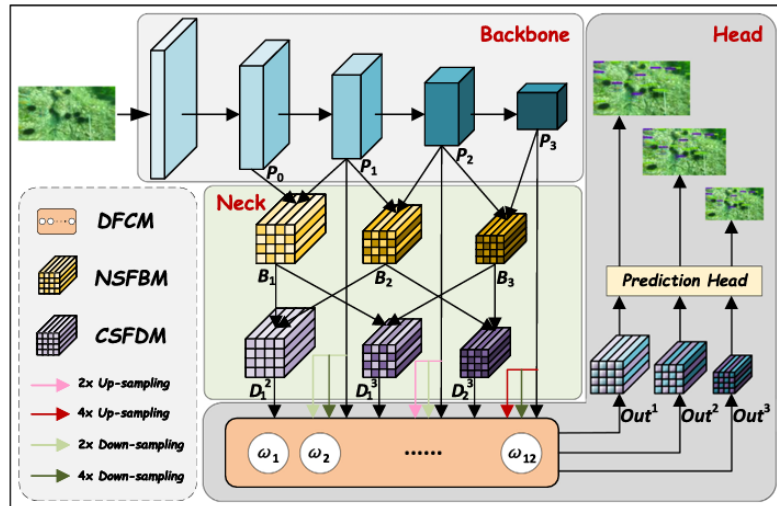


Figure 2.5: FBDPN Architecture Overview

2.5.6 Choosing the Right Model: Trade-Offs and Application-Specific Considerations

The choice of an optimal deep learning model for underwater object detection is highly dependent on the intended application context, particularly with respect to the trade-offs between inference speed, detection accuracy, and model complexity.

Referring to the model comparison table C.1, for applications requiring real-time performance, such as live aquaculture monitoring or autonomous underwater vehicle (AUV) navigation, YOLO-based models (e.g., YOLOv8 and CEH-YOLO) are particularly well-suited due to their high-speed inference and compact architectures. CEH-YOLO additionally offers improved detection of small and camouflaged marine objects, making it ideal for environments with visual complexity.

In contrast, models such as Faster R-CNN and the Swin Transformer prioritize detection accuracy and feature richness, excelling in scenarios where precise object localization and recognition are critical. These models are particularly beneficial for research-focused applications—such as biodiversity monitoring or marine species classification—where computational latency is less of a constraint. However, their relatively low inference speeds make them less practical for real-time use without dedicated hardware acceleration.

EfficientNet offers a balanced alternative, providing moderate-to-high accuracy with high efficiency, making it suitable for edge deployments or embedded systems where computational resources are limited. Its scalab-

ility and energy-efficient design make it an appealing option for lightweight deployments in moderately demanding underwater scenarios.

Finally, FBDPN, a hybrid CNN-Transformer architecture, delivers a compromise between speed and accuracy. By leveraging both local spatial feature extraction (via CNN) and global contextual awareness (via Transformer modules), it is particularly well-suited for practical deployments that require robustness across a range of underwater environments, including those with occlusion, turbidity, and varying lighting conditions.

In summary, selecting a model involves balancing application-specific requirements: YOLO-based models for real-time deployment, Faster R-CNN and Swin Transformer for precision-oriented tasks, EfficientNet for constrained environments, and FBDPN for flexible, high-performance use in challenging underwater settings.

2.5.7 Future Trends in Underwater Object Detection Architectures

As research progresses, hybrid models that combine CNNs and Transformers are expected to become more prevalent, leveraging the spatial efficiency of CNNs with the global attention capabilities of Transformers. Additionally, multi-modal learning, integrating optical, sonar, and LiDAR data, could further enhance detection robustness in low-visibility conditions. Ultimately, the continuous evolution of deep learning architectures, particularly YOLO-based enhancements, transformer integration, and hybrid models, will define the future of underwater object detection in both scientific research and commercial applications.

2.6 Future Directions

The field of underwater object detection using deep learning is evolving rapidly, propelled by advancements in image enhancement, model architecture innovations, and the expansion of specialized datasets. As researchers strive to overcome the inherent challenges of underwater environments such as low contrast, light distortion, and habitat variability several promising directions for future research are emerging.

2.6.1 Advanced Image Enhancement Techniques

While current image enhancement methods improve underwater visual quality, they often introduce artifacts that negatively impact detection performance. Future research will focus on developing adaptive, feature-preserving enhancement techniques.

One of those is designing unified models that perform image enhancement and object detection jointly within a single architecture. Such end-to-end systems would allow the model to optimize enhancement specifically for detection performance, minimizing the risk of feature extraction degradation introduced by separate preprocessing stages. This integration can significantly improve robustness under varying underwater visual conditions.

Another thing that can help improve images is by developing enhancement methods that explicitly model underwater light propagation physics, accounting for scattering, absorption, and color shift effects. By integrating physical principles into deep learning models, restoration can more accurately recover true object features and colors, leading to improved visibility, better feature preservation, and enhanced reliability for subsequent detection tasks.

Lastly, combining generative adversarial networks (GANs), attention mechanisms, and self-supervised learning strategies to achieve high-quality image restoration. These hybrid models aim to improve underwater visibility without compromising structural details critical for object detection. They could enable the creation of more naturalistic enhancements, better aligning visual improvement with real-world object recognition needs.

2.6.2 Evolution of Model Architectures

Deep learning architectures must continue to adapt to underwater-specific challenges, such as low contrast, small object size, and varying habitat conditions.

One of the promising directions is extending existing YOLO frameworks, such as CEH-YOLO, by integrating deformable attention modules and spatial pyramid pooling structures. These enhancements focus on improving the detection of small and partially occluded objects, common in cluttered underwater environments, while maintaining real-time inference speed crucial for autonomous underwater vehicle (AUV) deployment.

Other than that, expanding the adoption of Swin Transformers and Vision Transformers (ViTs) to leverage their global self-attention mechanisms.

Transformers are particularly promising for underwater imagery due to their ability to model long-range spatial dependencies, enabling better detection of camouflaged species and objects embedded within complex marine backgrounds where local CNN filters may struggle.

Last but not least, developing hybrid models that combine convolutional operations for efficient local feature extraction with transformer layers for capturing global contextual information. This hybrid approach can balance detection accuracy with computational efficiency, making models more practical for real-time underwater operations without sacrificing the nuanced feature extraction needed for fine-grained species classification.

2.6.3 Dataset Expansion and Cross-Domain Learning

A major limitation in underwater detection is dataset bias, where models trained on one environment struggle to generalize to others. Future dataset development should focus on dataset diversity, the use of sythetic dataset, improve on dataset annotation process, as well as incorporating cross-domain adaptation.

The rationale behind dataset ecological diversity is that creating more diverse datasets that encompass a wide range of aquatic environments, including freshwater, deep-sea, estuarine, and coastal habitats can expose models to varying lighting, turbidity, and species composition, leading to more generalizable detection performance.

Future dataset expansion should also focus on synthetic dataset generation using simulation frameworks to augment real-world data. Simulated images can replicate complex underwater conditions such as colour distortion, occlusion, and particulate matter that are challenging to capture consistently in field settings.

Addressing annotation challenges inherent to underwater datasets is one of the bigger challenges that should be tackled. Variability in visibility, motion blur, and camouflage often makes manual annotation difficult and time-consuming. Approaches such as semi-supervised labeling, active learning, and AI-assisted annotation tools could help streamline the labeling process and improve annotation accuracy across diverse environments.

Lastly, the use of cross-domain adaptation techniques, such as unsupervised domain adaptation (UDA), can bridge the performance gap between training and deployment domains. These techniques enable models to transfer learned features from one domain to another without requiring extensive labeled data from each new environment.

2.7 Conclusion

The ongoing advancements in deep learning are making underwater object detection more accurate, efficient, and adaptable, unlocking new applications in marine biodiversity conservation, aquaculture management, and underwater robotics. By integrating advanced image enhancement, optimized model architectures, and diverse datasets, researchers can develop next-generation detection systems capable of performing in diverse and challenging underwater environments. As deep learning continues to evolve, the future of underwater object detection holds immense potential, offering valuable insights into the hidden complexities of aquatic ecosystems.

While significant strides have been made in underwater object detection through deep learning advancements, the effectiveness and generalization of these models remain intrinsically tied to the quality and diversity of the datasets used for training. A model's success depends not only on architectural innovations but also on careful dataset selection, preprocessing, and augmentation strategies that expose the model to real-world underwater complexities.

Having established a strong foundation through the literature review, the next chapter outlines the research methodology. It details the dataset selection, preprocessing steps, model implementation, training configurations, and evaluation procedures that underpin the experimental framework used in this study.

3 Methodology

This chapter outlines the methodological framework adopted to develop, train, and evaluate deep learning models for underwater fish species detection and classification. It details the dataset selection and preprocessing steps undertaken, the model architectures chosen, the training procedures implemented, and the evaluation metrics applied. The approach was designed to ensure rigorous experimentation, reproducibility, and fairness in comparing different model performances.

3.1 Data Collection and Preprocessing

This section details the datasets that were considered to be used in the project and the preprocessing of the selected dataset. The transformation of the selected *Fish4Knowledge* dataset out of all collected datasets into a structure compatible with the YOLOv8 object detection framework is thoroughly discussed, as the dataset was originally designed for fish recognition using binary masks, and required significant preprocessing to adapt it for bounding box-based training.

3.1.1 Dataset Consideration and Selection

Before committing to a specific dataset for the object detection task, several publicly available fish-related datasets were explored and evaluated based on their suitability for training deep learning models such as YOLOv8, EfficientNet, and Swin Transformer. The selection criteria focused on dataset structure, annotation quality, species diversity, and real-world relevance to underwater imagery.

DeepFish Dataset

The *DeepFish* dataset, developed by Commonwealth Scientific and Industrial Research Organisation (CSIRO) and hosted on GitHub [15], was

one of the primary candidates considered for this project prior to selecting Fish4Knowledge. This dataset is well-structured and organized into three distinct subfolders, each corresponding to a specific computer vision task: classification, localization, and segmentation.

Of particular relevance to object detection is the segmentation folder, which contains paired image and binary mask files. This structure is notably similar to that of the Fish4Knowledge dataset and suggests the potential to extract bounding boxes from the masks to generate annotations suitable for YOLO-based training. The presence of binary masks for fish silhouettes enables the same mask-to-bounding-box conversion strategy previously applied, making the dataset technically compatible with the required pre-processing workflow.

While DeepFish offers a clean and structured format with real underwater imagery, a key limitation at the time of selection was its lack of consistent species-level labels, especially in the localization and segmentation subsets. Most annotations are limited to a generic "fish" class rather than specific species identifiers, reducing its immediate utility for training species-specific object detection models.

Nevertheless, due to its quality, structure, and real-world relevance, DeepFish remains a viable candidate for future work. It may be incorporated in subsequent phases of the project, particularly if additional labeled training data is required or if the objective shifts toward species-agnostic fish detection and segmentation.

A Large Scale Fish Dataset

Another dataset reviewed during the initial research phase was the *Large-Scale Fish Dataset* published by Ulucan et al. (2020) on Kaggle [16]. This dataset contains 9 different seafood classes, including commonly consumed species such as sea bass, trout, red mullet, and shrimp. It consists of 2000 images per class—1000 RGB images and 1000 corresponding pixel-level ground truth masks—created through data augmentation techniques (e.g., flipping and rotation).

The dataset is well-suited for segmentation and classification tasks and was used to benchmark various feature extraction and classification algorithms in its original publication. However, it was ultimately not selected for use in this project because the dataset was captured under laboratory-like conditions using supermarket-sourced fish laid out for photography, rather than in natural or underwater environments. The species diversity is relatively limited, covering only 9 seafood types, with a focus on commercial food

species rather than ecological or in situ marine fauna. The dataset lacks object detection-style bounding box annotations, and while masks could theoretically be converted to boxes, the images' controlled settings reduce their applicability to real-world marine environments.

Given the project's focus on developing robust underwater detection models for use in unconstrained environments, this dataset was not deemed suitable for training or evaluation purposes.

NOAA Fisheries Dataset (Labeled Fishes in the Wild)

The *Labeled Fishes in the Wild* dataset, released by NOAA Fisheries [17], provides annotated underwater imagery for the development and evaluation of fish detection models. The dataset includes over 900 still images in the training set, each annotated with bounding boxes indicating the presence of fish. The test set comprises a short video sequence, simulating deployment in real-world aquatic environments.

While the dataset is well-curated in terms of visual quality and ecological relevance, its practical usability was limited due to the annotation format and the nature of the test data. Specifically, bounding box annotations are provided in OpenCV-specific formats, namely `.vec` and `.dat` files. These formats are primarily designed for use with OpenCV's Haar cascade framework and are not natively supported by modern deep learning libraries such as PyTorch or TensorFlow. As a result, utilizing these annotations would have required developing a custom parser or converter to extract bounding box information into a more standard format, such as COCO or YOLO.

Moreover, the dataset's test set consists exclusively of a video file without corresponding per-frame image annotations. To use this video in a supervised evaluation setting, one would need to extract individual frames and manually annotate them—an effort-intensive task that was not feasible within the scope and timeframe of this project.

Due to these technical limitations and resource constraints, the *Labeled Fishes in the Wild* dataset was not selected for use in this study.

WildFish Dataset

The *WildFish* dataset, developed by Zhuang et al. (2023) [18], is one of the most comprehensive large-scale fish datasets currently known. It contains over 200,000 images across 1,960 fish species, annotated with

both bounding boxes and species-level labels. The dataset presents a significant opportunity for training robust deep learning models for fish classification and object detection in real-world conditions.

However, despite its technical merits, the dataset was ultimately deemed unsuitable for this project due to significant accessibility and logistical barriers. Specifically, the dataset was not made publicly available via direct download. Instead, access required the use of a third-party file transfer application, which in turn necessitated registration using a valid Chinese phone number — a restriction that significantly limits international academic usability.

Upon contacting the original authors to request an alternative download method, the dataset was provided in the form of 18 compressed .zip folders. Collectively, these files amounted to approximately 20 terabytes (TB) of storage, far exceeding the storage and processing capabilities allocated for this project. Given these constraints, and in the interest of ensuring reproducibility and feasibility within the scope of this dissertation, the WildFish dataset was excluded from further consideration.

Rationale for Selecting Fish4Knowledge

The *Fish4Knowledge* dataset was developed as part of a large-scale European research project aimed at advancing automatic analysis of underwater video for marine ecology. Curated by researchers at the University of Edinburgh, the dataset includes manually annotated ground truth data for species recognition collected from public underwater cameras located around Taiwan [19]. It provides thousands of labeled images across various fish species and is designed to support tasks such as object detection, tracking, and classification in challenging underwater environments. The dataset has become a foundational benchmark for evaluating underwater computer vision systems.

After evaluating several candidate datasets, the Fish4Knowledge dataset was selected for this project based on its alignment with the goals of species-specific fish detection in underwater environments. The dataset offers several key advantages over other publicly available datasets:

- **Species Diversity:** The dataset includes 27,370 images of verified fish and 23 distinct fish species, each organized into its own subdirectory, providing a structured and although not well-balanced distribution of classes for training, this can be mitigated by randomly splitting it to train, validate and test set.

- **Paired Mask Annotations:** Each fish image is accompanied by a binary segmentation mask, allowing for the extraction of precise bounding boxes. This facilitates the conversion of mask data into YOLO-compatible annotation format.
- **Underwater Realism:** All images were collected from fixed underwater camera systems deployed in marine environments, ensuring ecological validity and robustness for downstream object detection tasks.
- **Reusability and Structure:** The dataset is well-organized and suitable for programmatic parsing, with consistent file naming and logical folder hierarchy. This enables efficient preprocessing and integration into modern deep learning pipelines.
- **Public Accessibility:** Unlike some datasets with access restrictions or licensing constraints, Fish4Knowledge is freely available for academic use, facilitating reproducibility and collaboration.

These factors collectively make Fish4Knowledge the most appropriate choice for this study. Its structural richness, annotation quality, and ecological relevance provide a strong foundation for training and evaluating deep learning models for fish species detection in realistic underwater settings.

Table C.2 provides a comparative summary of all datasets evaluated during the selection process.

3.1.2 Dataset Preprocessing

Following the selection of the Fish4Knowledge dataset, a series of preprocessing steps were necessary to adapt it for object detection and classification tasks. This subsection provides an overview of the dataset structure, discusses the motivation behind adapting it, outlines the preprocessing workflow, and describes the resulting dataset format used for model training and evaluation.

Overview of Fish4Knowledge Dataset

The Fish4Knowledge dataset was obtained from the University of Edinburgh’s official repository [19]. It consists of underwater fish imagery annotated using binary segmentation masks. The dataset is structured into two parallel directories:

```
Fish4Knowledge/  
  fish_image/  
    fish_01/  
    fish_02/  
    fish_03/  
  mask_image/  
    mask_01/  
    mask_02/  
    mask_03/
```

- `fish_image/` — contains subdirectories `fish_01` through `fish_23`, each corresponding to a distinct fish species. Each subfolder contains multiple cropped fish images in `.png` format.
- `mask_image/` — mirrors the species-based folder structure of `fish_image/`, with binary mask images in `.png` format representing fish silhouettes for each image.

Additionally, a `class_id.csv` file that needed to be downloaded separately, maps image filenames to their corresponding species identifiers.

Motivation for Dataset Adaptation

Although Fish4Knowledge offers well-organized image and mask data, it is incompatible with object detection frameworks such as YOLOv8, which require bounding box annotations in a specific format. YOLO expects:

- Class labels and bounding box coordinates in normalized form.
- A dedicated label file for each image in `.txt` format.
- Dataset partitioned into `train`, `val`, and optionally `test` subsets.

Therefore, a complete preprocessing pipeline was required to:

1. Convert mask annotations into bounding boxes.
2. Normalize bounding boxes and format them for YOLO.
3. Split the dataset into appropriate subsets.
4. Ensure compatibility of image-label pairs for training.

Dataset Processing Workflow

A custom Python script was implemented to automate the entire preprocessing pipeline. The steps are summarized below:

1. **Annotation generation:** Each fish image was matched with its corresponding binary mask. OpenCV's `findContours()` and `boundingRect()` functions were used to compute bounding boxes around the fish region.
2. **Conversion to YOLO format:** The bounding boxes were normalized relative to image width and height and saved as `.txt` files. Each annotation followed the YOLO format:

$$\langle \text{class_id} \rangle \langle x_{center} \rangle \langle y_{center} \rangle \langle width \rangle \langle height \rangle$$

3. **Dataset splitting:** Image-label pairs were randomly assigned to `train` (70%), `val` (20%), and `test` (10%) directories.
4. **Visual validation:** A preview folder was generated containing a subset of images with annotated bounding boxes overlaid. This allowed manual verification of annotation correctness.

Resulting Dataset Structure

After execution of the preparation pipeline, the dataset was restructured into a YOLOv8-compatible format as follows:

```
Fish4K-YOLO/  
  images/  
    train/  
    val/  
    test/  
  labels/  
    train/  
    val/  
    test/  
  preview/  
  data.yaml
```

Each image in the `images` folder has a corresponding annotation file in the `labels` directory, both sharing the same base filename. The `preview/`

directory contains visually annotated images used for quality assurance as shown in Figure B.1. The `data.yaml` file specifies dataset paths and class label mappings for use in training scripts.

In total, after all preprocessing was conducted, the dataset contained 19,159 images for training, 5,474 images for validating, and 2,737 images for testing as can be seen in the terminal screenshot of Figure B.2. Providing around 70%-20%-10% training, validating and testing split, respectively.

Through this pipeline, the Fish4Knowledge dataset was successfully converted into a YOLOv8-ready format. The structured annotations and visual validation process ensured high-quality inputs for downstream object detection tasks. This preprocessing stage plays a critical role in enabling effective model training and performance evaluation.

Having successfully prepared the Fish4Knowledge dataset for training and evaluation, the next step involves selecting and implementing suitable deep learning models capable of performing underwater fish detection and classification with high accuracy. The following section presents the models chosen for this study and outlines their respective training configurations.

3.2 Model Selection and Implementation

Building on the preprocessed dataset, this study evaluates the effectiveness of three state-of-the-art deep learning architectures: **YOLOv8**, **EfficientNet**, and the **Swin Transformer**. Each model represents a distinct class of neural network architectures—ranging from lightweight, real-time object detectors to high-performance convolutional and transformer-based classifiers.

The rationale behind selecting these models is grounded in their complementary strengths:

- **YOLOv8** serves as a real-time, anchor-free object detection model capable of detecting and localizing multiple fish instances per image. It is especially well-suited for detection tasks involving spatially diverse underwater scenes [20].
- **EfficientNet** is employed as a high-performing convolutional neural network for image classification. Its compound scaling approach makes it both accurate and computationally efficient, making it ideal for species-level classification of cropped or isolated fish images [21].
- **Swin Transformer** introduces a hierarchical vision transformer archi-

texture that leverages shifted windows for local and global attention. Its ability to model long-range dependencies is particularly valuable for interpreting complex underwater imagery where context and background may influence predictions [22].

By evaluating these three architectures under the same dataset and conditions, the study aims to compare their performance across multiple dimensions including:

- Detection accuracy
- Classification precision
- Model generalizability
- Inference speed
- Suitability for underwater applications

Each model will be trained using the preprocessed Fish4Knowledge dataset, with the goal of identifying which architecture offers the best trade-off between accuracy, computational cost, and real-world applicability in marine environments.

3.2.1 YOLOv8 Model

The first model selected for evaluation is YOLOv8, a state-of-the-art object detection framework renowned for its balance between speed and accuracy. As a real-time detection model, YOLOv8 provides an important benchmark for assessing the feasibility of rapid underwater species identification.

Architecture

YOLOv8 is the eighth and most recent version of the "You Only Look Once" (YOLO) family of object detection models, developed by Ultralytics. It is designed to be fast, lightweight, and accurate, with significant architectural upgrades over its predecessors [20].

One of the most defining features of YOLOv8 is its shift to an **anchor-free detection mechanism**. Unlike earlier YOLO versions that used predefined anchor boxes to propose object regions, YOLOv8 eliminates the need for anchors, simplifying the learning process and improving detection performance, especially for small or densely packed objects.

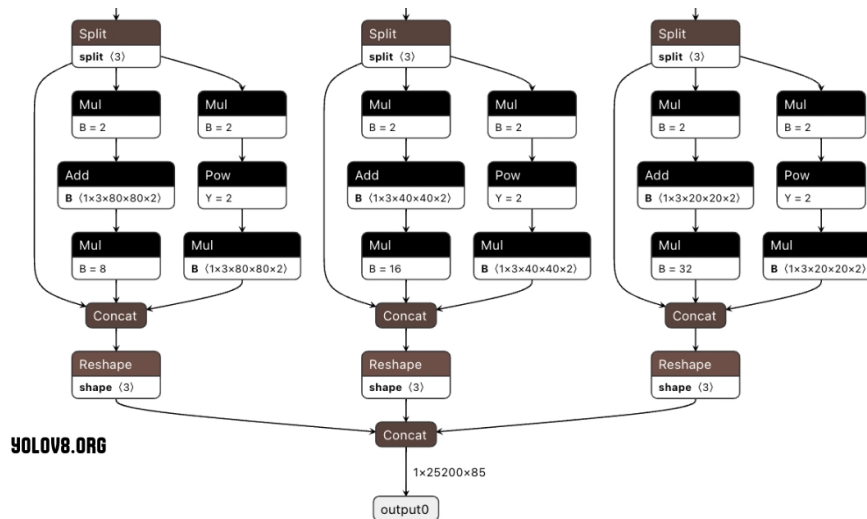


Figure 3.1: YOLOv8 output head architecture for multi-scale detection

The model uses a single-stage detection pipeline, allowing it to predict bounding box coordinates and class probabilities simultaneously in a single forward pass. This contributes to its real-time inference speed, making it suitable for edge and embedded deployments.

YOLOv8 adopts a modular architecture composed of three main components:

- **Backbone:** A modified version of the CSPDarknet backbone is used to extract high-level spatial and semantic features from the input image.
- **Neck:** The model uses a Path Aggregation Network (PAN) structure to combine feature maps from different layers, enhancing the model's ability to detect objects at various scales.
- **Head:** The detection head is responsible for producing final predictions, including bounding box coordinates, objectness scores, and class labels. It incorporates a Distribution Focal Loss (DFL) layer to improve bounding box regression accuracy.

The Figure 3.1 shows the multi-scale processing of feature maps at three resolutions (80x80, 40x40, 20x20), where outputs are refined and merged into a unified prediction tensor of shape 1x25200x85. This structure enables multi-scale object detection, allowing YOLOv8 to detect both small and large objects efficiently within a single forward pass.

YOLOv8 also introduces a number of enhancements that contribute to improved performance:

- **Decoupled head design**, which separates objectness prediction from classification and regression tasks.
- **Improved activation functions** (such as SiLU) and normalization layers for more stable convergence.
- **Exportable formats**, allowing the model to be deployed easily in ONNX, CoreML, TensorRT, and other formats.

These innovations enable YOLOv8 to achieve superior performance in both speed and accuracy, while maintaining flexibility across different detection scenarios and hardware platforms.

Training Setup and Parameters

The YOLOv8 model was trained using the Ultralytics implementation with the `yolov8n.pt` pretrained weights [23]. The training process employed a transfer learning approach, where a subset of layers from the pretrained model was fine-tuned on the Fish4Knowledge-YOLO dataset. Specifically, 319 out of 355 parameters were transferred, while some layers related to the distribution focal loss head were frozen.

To improve computational efficiency and reduce memory usage, **Automatic Mixed Precision (AMP)** training was enabled, with AMP checks successfully passed prior to training.

The dataset was divided into training and validation sets, containing:

- **Training set:** 19,159 images
- **Validation set:** 5,474 images

All input images were automatically resized to a resolution of 640×640 pixels during training and inference. The model was trained for a total of **50 epochs** (Originally 100 epochs but decided it was too long), using a batch size of **16** images and **8 dataloader workers** for parallel processing.

The following training parameters were used:

- **Optimizer:** Stochastic Gradient Descent (SGD) was employed as the optimizer. SGD is a widely used optimization algorithm in deep learning due to its simplicity and effectiveness in finding minima of the loss function, particularly for large datasets. In this setup, SGD was used with automatic adjustment of the learning rate and momentum to improve training efficiency and stability.

- **Learning Rate:** The learning rate was initialized at 0.01, serving as the step size for updating model parameters. A proper learning rate is critical: if set too high, it can cause divergence; if too low, it can result in slow convergence. Automatic warm-up and scheduling were enabled to dynamically adjust the learning rate during early training stages and based on performance, facilitating smoother and faster convergence.
- **Momentum:** A momentum value of 0.9 was used to accelerate the optimizer in the relevant direction and dampen oscillations. Momentum helps the optimizer overcome small local minima and reduces variance in parameter updates, leading to faster and more stable convergence.
- **Weight Decay:** A weight decay value of 0.0005 was applied to weight parameters to penalize large weights during training, thereby acting as a regularization technique. This helps in reducing overfitting by encouraging the model to learn simpler representations. No decay was applied to bias parameters, following standard practices to maintain flexibility in bias terms.
- **Loss functions:** YOLOv8 employs a composite loss function designed to optimize both object classification and localization accuracy. The total loss consists of three components: bounding box regression loss ($\mathcal{L}_{\text{box}}$), object classification loss ($\mathcal{L}_{\text{cls}}$), and distribution focal loss ($\mathcal{L}_{\text{dfl}}$), the latter of which improves bounding box precision [24]. The complete loss function is expressed as:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{box}} + \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{dfl}}$$

- **Bounding Box Loss ($\mathcal{L}_{\text{box}}$):** Bounding box regression is handled using the Complete Intersection over Union (CIoU) loss [25], which combines IoU score, center point distance, and aspect ratio differences:

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v$$

where ρ is the Euclidean distance between the centers of the predicted and ground truth boxes, c is the diagonal length of the smallest enclosing box, v is the aspect ratio term, and α is a balancing factor.

- **Classification Loss ($\mathcal{L}_{\text{cls}}$):** YOLOv8 uses Binary Cross Entropy (BCE) loss for multi-label classification. For a single class

prediction, the BCE formula is:

$$\mathcal{L}_{\text{BCE}} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

where y is the binary ground truth label, and $\hat{y}$ is the predicted probability. This formulation allows the model to handle overlapping object classes in a multi-label setting.

- **Distribution Focal Loss** ($\mathcal{L}_{\text{dfl}}$): Unlike standard Focal Loss used for classification in some detectors, YOLOv8 applies Distribution Focal Loss only to improve bounding box regression [26]. DFL models bounding box coordinates as discrete distributions across regression bins to enhance precision. The loss is applied as a soft classification over the predicted bins and their corresponding targets:

$$\mathcal{L}_{\text{dfl}} = \sum_{i=1}^N \text{CE}(p_i, q_i)$$

where p_i is the predicted probability distribution, q_i is the ground truth bin label, and CE denotes cross-entropy loss.

This carefully balanced loss formulation enables YOLOv8 to learn robust object localization and classification in a single-stage detector framework, especially under varying object scales and scene complexities.

Evaluation Metrics

The YOLOv8 model was evaluated using several standard metrics commonly applied in object detection tasks. These metrics provide comprehensive insight into the model's classification accuracy and localization quality. Each metric is described below.

Precision measures the proportion of true positive predictions among all positive predictions made by the model. High precision indicates a low false positive rate, meaning the model's positive predictions are reliable. Precision is particularly important in contexts where false positives carry significant costs or consequences, such as detecting rare fish species or critical events. The equation for this is as below:

$$\text{Precision} = \frac{TP}{TP + FP}$$

where:

- TP : True Positives (correct detections)
- FP : False Positives (incorrect detections)

Recall, also known as sensitivity, quantifies the proportion of actual positives that were correctly identified by the model. High recall suggests the model is effective at detecting most instances of the target class, minimizing false negatives. Recall is especially crucial in applications where missing a positive instance would be more detrimental than a false alarm. Below is the equation for recall:

$$\text{Recall} = \frac{TP}{TP + FN}$$

where:

- FN : False Negatives (missed detections)

The **F1-score** represents the harmonic mean of precision and recall. It balances the trade-off between false positives and false negatives into a single metric. The F1 score is particularly useful when dealing with imbalanced datasets, as it does not favor either precision or recall but instead seeks to optimize both simultaneously. F1-score equation can be referred to as:

$$\text{F1-score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

To avoid numerical instability when both precision and recall are close to zero, a small constant ε may be added:

$$\text{F1-score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall} + \varepsilon}, \quad \text{where } \varepsilon \ll 1$$

Average Precision (AP) is the area under the precision-recall curve for a specific class and Intersection over Union (IoU) threshold. This is important as it is used to compute the mAP@0.5 and others, which are the standard metrics to evaluate object detection performance instead of the classic accuracy:

$$\text{AP} = \int_0^1 \text{Precision}(\text{Recall}) d\text{Recall}$$

This integral is typically approximated using numerical methods such as 101-point interpolation.

Mean Average Precision at an Intersection over Union (IoU) threshold of 0.5 (**mAP@0.5**) is a widely used object detection metric. It measures the average precision across all classes when a predicted bounding box is considered correct if it overlaps with the ground truth box by at least 50%. A higher mAP@0.5 score indicates more accurate object localization and classification. The equation is defined as:

$$\text{mAP@0.5} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i^{0.5}$$

where:

- N : Total number of classes
- $\text{AP}_i^{0.5}$: Average Precision for class i at IoU = 0.5

The **mAP@0.5:0.95 (mAP@0.5:0.95)** metric averages the mean Average Precision over multiple IoU thresholds, ranging from 0.5 to 0.95 in steps of 0.05. This stricter metric demands not only correct detections but also precise localization. It provides a more comprehensive assessment of the model's overall detection performance across varying levels of localization difficulty. It is mathematically defined as:

$$\text{mAP@0.5:0.95} = \frac{1}{10} \sum_{t=0.5}^{0.95} \text{mAP@t}$$

where $t \in \{0.5, 0.55, 0.60, \dots, 0.95\}$ represents different IoU thresholds.

The **Confusion Matrix** is a tabular summary that illustrates the number of correct and incorrect predictions for each class. Each row of the matrix represents instances in a predicted class, while each column represents instances in an actual class. A perfect classification would result in all predictions lying along the matrix's diagonal. The confusion matrix enables the identification of specific classes where misclassifications are more frequent.

For object detection tasks, including YOLOv8, standard evaluation metrics such as Precision, Recall, mAP@0.5, and mAP@0.5:0.95 are employed instead of simple accuracy, as detection involves both classification and localization of objects.

During training, all metrics including losses and validation performance were logged and visualized in the `runs/yolov8_fish4k_eval` directory. The training was conducted using a GPU-enabled machine to accelerate convergence and allow large-scale parallelism.

As training progressed, notable improvements were observed in both bounding box accuracy and classification performance, as illustrated in the evaluation metrics described in Chapter 4.

Dataset Preparation

The base dataset used in this study originates from the Fish4Knowledge collection, which is discussed in detail in Section 3.1.2 and beyond. That section covers the dataset's class organization, original structure, and motivation for adaptation.

For the purpose of training YOLOv8, the dataset required additional transformation steps to convert it into a format compatible with YOLO-style object detection. This involved:

- **Mask-to-Bounding Box Conversion:** Binary masks were processed using contour detection to extract the fish's bounding box. The bounding box coordinates were normalized relative to image dimensions.
- **YOLO Label Formatting:** Each image's annotation was saved in YOLO format — as a text file containing the class ID and the normalized coordinates of the bounding box center, width, and height.
- **Dataset Organization:** The dataset was organized into YOLOv8's expected structure, with separate subfolders for images and labels under `train`, `val`, and `test`. A `data.yaml` file was created to define class names and file paths.
- **Data Augmentation:** No custom or manual data augmentation techniques were applied during preprocessing. However, YOLOv8 applies its own built-in augmentations during training, including random flipping, scaling, translation, and mosaic augmentation, which were left enabled by default.
- **Visual Validation:** A sample of the generated labels was overlaid on the original images to confirm alignment between masks and bounding boxes.

These preprocessing steps ensured that the Fish4Knowledge dataset could be used effectively in YOLOv8's training pipeline without manual annotation or structural inconsistencies. All of the processes that were undertaken can be seen in the script `prepare_fish4k_yolo.py` file.

Training Procedure

The YOLOv8 model was trained on a local machine equipped with an AMD Ryzen 7 6800HS processor (3.20 GHz base clock), 64 GB of RAM, and a dedicated Nvidia GeForce RTX3070 Ti Laptop GPU. The system ran on a 64-bit Windows 11 Pro operating system (version 24H2), with support for GPU acceleration through integrated Radeon Graphics. The training environment was built using Python 3.13, and core libraries included Ultralytics, PyTorch, NumPy, OpenCV, and Matplotlib. All dependencies were managed and version-controlled using an `environment.yml` file to ensure reproducibility.

Training was executed using the YOLOv8n (nano) model initialized with pretrained weights (`yolov8n.pt`). The training process used default YOLOv8 settings, which is summarized in Table 3.1.

Table 3.1: YOLOv8 Training Configuration

Aspect	Details
Model Architecture	YOLOv8n (Nano) pre-trained on COCO
Transfer Learning	Partial transfer (319/355 layers)
Training Duration	50 epochs, ~4 minutes per epoch
Input Image Size	640 × 640 pixels
Optimizer	Stochastic Gradient Descent (SGD)
Learning Rate	0.01 (auto-tuned)
Momentum	0.9
Weight Decay	0.0005
Loss Functions	CloU Loss (box), BCE Loss (class), DFL Loss (box precision)
Batch Size	16 (8 dataloader workers for parallel processing)
Data Augmentation	Default YOLOv8 augmentations (Mosaic, HSV shift, random flip)
Precision Mode	Automatic Mixed Precision (AMP) enabled

The model was trained for **50 epochs**, with each epoch taking approximately **4 minutes** to complete, leading to a total training time of about **3 hours**. The training logs, and model checkpoints were stored automatically in the `runs/yolov8_fish4k_train` directory. While the evaluation metrics and plot diagrams were stored in `runs/yolov8_fish4k_eval`.

The training was monitored using YOLOv8's built-in logging system, which tracked key performance metrics including training and validation losses, mean Average Precision (mAP), and learning rate progression. Early

epoch outputs confirmed that the model was successfully fine-tuning on the Fish4Knowledge-YOLO dataset using transferred weights and default augmentation strategies which can be referred to in Section 4.1.1.

3.2.2 EfficientNet Model

Following the YOLOv8 object detection framework, EfficientNet is implemented as a dedicated image classification model. EfficientNet is particularly notable for its compound scaling method, offering strong performance across a range of computational budgets, making it highly relevant for underwater species recognition tasks.

Architecture

EfficientNet is a family of convolutional neural network (CNN) architectures introduced by Google AI, designed to achieve state-of-the-art performance while maintaining computational efficiency. It is known for its systematic model scaling method, which balances network depth, width, and resolution using a compound scaling coefficient [21].

The architecture of EfficientNet-B0 can be broadly divided into three main components: the stem, the body, and the head.

The **stem** serves as the initial feature extraction layer. It consists of:

- A standard convolutional layer with 32 filters.
- Kernel size of 3×3 , with stride 2.
- Followed by batch normalization.
- ReLU6 activation function applied after normalization.

The purpose of the stem is to quickly downsample the input image while extracting low-level features.

The **body** forms the core of the EfficientNet-B0 architecture and is composed of multiple stages of **MBConv blocks** (Mobile Inverted Bottleneck Convolutions). Each MBConv block typically includes:

- **Expansion Layer:** Expands the number of channels using a 1×1 convolution based on a defined expansion ratio.
- **Depthwise Separable Convolution:** A lightweight convolution oper-

ation applied channel-wise, reducing computation.

- **Squeeze-and-Excitation (SE) Block:** Adaptively recalibrates channel-wise feature responses by modeling interdependencies between channels.
- **Projection Layer:** Reduces the expanded channels back to a lower-dimensional output.

Important parameters that vary across different MBConv stages include:

- **Expansion ratio:** The factor by which input channels are expanded.
- **Kernel size:** The convolutional filter size (typically 3×3 or 5×5).
- **Stride:** Determines spatial downsampling (typically 1 or 2).
- **SE ratio:** Ratio for the squeeze operation in SE blocks (commonly 0.25).

The head aggregates the features extracted by the body and produces final class probabilities. It consists of:

- A final 1×1 convolutional layer.
- Global average pooling layer that reduces the spatial dimensions to a single vector.
- A fully connected (dense) layer for classification.
- Softmax activation function applied to produce normalized class probabilities.

This structured architecture as summarized in Figure 3.2, combined with compound scaling, allows EfficientNet-B0 to achieve state-of-the-art performance with fewer parameters and lower FLOPs compared to traditional CNNs [27]. In this project, EfficientNet-B0 was modified by replacing the original final classification layer to match the 23 fish species classes from the Fish4Knowledge dataset.

The key innovation behind EfficientNet lies in its use of **compound scaling**, where all three dimensions of the network (depth d , width w , and resolution r) are scaled uniformly using a set of fixed coefficients. This is in contrast to traditional approaches that scale one dimension at a time, often leading to suboptimal performance. The compound scaling approach is formulated as:

Stage	Operator	Resolution	#Channels	#Layers
1	Conv3x3	224 × 224	32	1
2	MBConv1, k3x3	112 × 112	16	1
3	MBConv6, k3x3	112 × 112	24	2
4	MBConv6, k5x5	56 × 56	40	2
5	MBConv6, k3x3	28 × 28	80	3
6	MBConv6, k5x5	14 × 14	112	3
7	MBConv6, k5x5	14 × 14	192	4
8	MBConv6, k3x3	7 × 7	320	1
9	Conv1x1 & Pooling & FC	7 × 7	1280	1

Figure 3.2: Summary of EfficientNet-B0 Architecture

$$\text{depth} \propto \alpha^\phi, \quad \text{width} \propto \beta^\phi, \quad \text{resolution} \propto \gamma^\phi$$

subject to the constraint: $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$, where ϕ is the user-specified compound coefficient that controls overall model size.

EfficientNet uses **Mobile Inverted Bottleneck Convolution (MBConv)** blocks, first introduced in MobileNetV2, combined with **Squeeze-and-Excitation (SE)** blocks for channel-wise attention. The architecture is also built on the Swish activation function (a smooth, non-monotonic alternative to ReLU) and optimized batch normalization techniques.

The base model, EfficientNet-B0, is designed using Neural Architecture Search (NAS), and larger variants (B1 to B7) are generated using the compound scaling formula. Each successive variant increases performance at the cost of more computation.

Key architectural features include:

- Depthwise separable convolutions for efficient computation
- MBConv blocks with residual connections
- Squeeze-and-Excitation (SE) layers to recalibrate feature maps

- Swish activation for better gradient flow

EfficientNet has been shown to outperform many traditional CNNs (e.g., ResNet, Inception) while using significantly fewer parameters and FLOPs, making it suitable for real-time and mobile deployment in scenarios such as underwater species classification.

Training Setup and Parameters

The EfficientNet model was trained using a transfer learning approach, initializing weights from a pretrained EfficientNet-B0 model trained on the ImageNet dataset. Transfer learning enables faster convergence and better generalization, particularly when limited labeled data is available, by leveraging learned feature representations from a large-scale dataset. The final classification head was modified to output 23 classes, corresponding to the distinct fish species within the Fish4Knowledge dataset.

The selection of training parameters plays a crucial role in ensuring the stability, efficiency, and performance of deep learning models. Each hyperparameter, including the optimizer settings, input size, learning rate, batch size, and data augmentation strategy, has a direct impact on the model's ability to learn meaningful representations from the data without overfitting or underfitting. Therefore, careful tuning and justified selection of these values were necessary to maximize the EfficientNet model's performance on the Fish4Knowledge dataset. The following configuration was adopted for this study, with each component selected based on its alignment with best practices for transfer learning and constrained computational resources.

The specific training configuration was as follows:

- **Optimizer:** The AdamW optimizer was employed due to its effectiveness in training deep convolutional networks with improved handling of weight decay. AdamW decouples weight decay from the gradient update step, leading to better generalization compared to traditional Adam, particularly when regularization is important [28].
- **Learning Rate:** A fixed learning rate of 0.001 was chosen, balancing the need for sufficiently large updates early in training while maintaining stability. Learning rate warm-up or decay schedules were not applied initially to retain simplicity, given that the pretrained backbone already provided stable feature representations.
- **Weight Decay:** A weight decay of 0.0001 was applied to penalize large weights and reduce overfitting. This is particularly important in transfer learning to ensure that the fine-tuned model does not overfit

to the relatively smaller Fish4Knowledge dataset.

- **Batch Size:** A batch size of 32 was selected based on a balance between memory constraints and statistical stability in gradient estimation. Larger batch sizes were avoided due to limited GPU memory availability.
- **Number of Epochs:** The model was trained for 50 epochs. This number was empirically chosen to allow sufficient convergence, while avoiding excessive training that could lead to overfitting, as observed during early experiments.
- **Loss Function:** Cross-Entropy Loss was employed as the loss function to optimize the EfficientNet model. This choice is standard for multi-class single-label classification problems where each image belongs to exactly one class.

Mathematically, for a given input sample with ground truth class label $y \in \{1, 2, \dots, C\}$ and model predicted probability distribution $\hat{y} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_C]$, the Cross-Entropy Loss $\mathcal{L}_{CE}$ is defined as:

$$\mathcal{L}_{CE} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

where:

- C is the total number of classes (23 fish species in this case),
- y_i is a binary indicator (0 or 1) whether class i is the correct class label,
- $\hat{y}_i$ is the model's predicted probability for class i .

In practice, the label y is provided as an integer corresponding to the correct class, and the model outputs a probability vector $\hat{y}$ after applying the softmax function to its raw logits.

The intuition behind Cross-Entropy Loss is that it measures the divergence between the true distribution (where the correct class has probability 1) and the predicted probability distribution produced by the model. Minimizing this loss encourages the model to assign higher probability to the correct class and lower probabilities to all incorrect classes.

This loss function is particularly suitable for the fish species classification task, as each input image belongs to exactly one and only one

fish species, without multi-label overlap.

Evaluation Metrics

To comprehensively evaluate the performance of the EfficientNet model on the Fish4Knowledge dataset, a set of standard classification metrics was employed. These metrics assess the model's ability to correctly predict the fish species, taking into account both overall accuracy and the quality of predictions across different classes. The metrics used are described below.

Accuracy measures the proportion of correctly classified samples among the total number of samples. It is computed as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP represents true positives, TN true negatives, FP false positives, and FN false negatives. Accuracy provides an overall sense of the model's performance but can be misleading in the presence of class imbalance. Therefore, additional metrics were also evaluated.

Precision quantifies the proportion of positive identifications that were actually correct. For a specific class, it is defined as:

$$\text{Precision} = \frac{TP}{TP + FP}$$

High precision indicates that the model makes few false positive errors. Macro-averaged precision was used, treating each class equally regardless of its frequency.

Recall, also known as sensitivity, measures the proportion of actual positives that were correctly identified. It is given by:

$$\text{Recall} = \frac{TP}{TP + FN}$$

High recall implies that the model detects most of the true instances. Macro-averaged recall was computed to fairly assess performance across all fish species.

The **F1-score** is the harmonic mean of precision and recall, providing a single metric that balances both false positives and false negatives:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Macro-averaged F1-score was used in this study to ensure that minority classes were given equal importance during evaluation.

A **Normalized Confusion Matrix** was generated to visualize the model's classification performance across all 23 fish species. The confusion matrix displays the number of correct and incorrect predictions for each class, helping to identify classes that are frequently confused with one another. Normalization was performed to account for different numbers of samples per class, thereby facilitating easier comparison.

In addition to final evaluation metrics, the evolution of **training and validation loss and accuracy** was plotted over the course of training. These curves provide insights into the learning dynamics, convergence behavior, and potential signs of overfitting.

All evaluation metrics were computed using the `scikit-learn` library and visualization was performed with `seaborn` and `matplotlib`.

Dataset Preparation

The Fish4Knowledge dataset was originally organized for segmentation and recognition tasks, containing fish images and corresponding binary masks structured into species-specific folders. To adapt the dataset for image classification with EfficientNet, a systematic preprocessing and reorganization process was undertaken.

First, the original `fish_image/` directory, which contained subfolders named by species identifiers (e.g., `fish_01`, `fish_02`, etc.), was used as the source. Each subfolder exclusively contained images belonging to a single fish species, thus making it suitable for direct adaptation into a classification framework.

To prepare the dataset for training, the images were randomly split into three subsets:

- **Training set:** 70% of the available images from each species
- **Validation set:** 20% of the available images from each species
- **Test set:** 10% of the available images from each species

The splitting was performed stratified by class to ensure that all subsets

contained images from every species, thereby avoiding class omission during training or evaluation phases. Random shuffling was applied before splitting to prevent ordering biases.

Each split maintained a structured folder hierarchy following the conventions expected by PyTorch's `ImageFolder` class:

```
Fish4K-Split/  
  train/  
    fish_01/  
    fish_02/  
    ...  
  val/  
    fish_01/  
    fish_02/  
    ...  
  test/  
    fish_01/  
    fish_02/  
    ...
```

This reorganization allowed for seamless loading of the data using standard classification dataloaders, where each subfolder name is mapped to a class label automatically.

In addition to the main dataset splits, a small set of preview images was generated from each split to visually verify the success of the reorganization and check for correct class distribution. Furthermore, a class distribution summary was saved into a text file, recording the number of images per species for train, validation, and test splits, thus ensuring reproducibility and transparency in dataset preparation. All of the processes that were undertaken can be seen in the script `prepare_fish4k_efficientnet.py` file.

Training Procedure

Training was conducted on a local machine equipped with an AMD Ryzen 7 6800HS processor (3.20 GHz base clock) and 64 GB of RAM, running Windows 11 Pro (64-bit, version 24H2), and an Nvidia GeForce RTX3070 Ti Laptop GPU. GPU acceleration through the dedicated graphics card was utilized to expedite training. The environment was managed via Python 3.13, using key libraries such as PyTorch, torchvision, scikit-learn, NumPy, and Matplotlib. Dependency versions were managed through a

Conda environment, specified in an `environment.yml` file to ensure reproducibility.

The EfficientNet model was trained for fish species classification following a structured and reproducible pipeline. The training was conducted using the Fish4K-Split dataset, with training, validation, and test splits preorganized to match standard classification loading formats.

During training, data augmentations were applied to the training set dynamically at each epoch. Augmentations included random resized cropping, horizontal flipping, slight rotation, and colour jittering, simulating the natural variability present in underwater environments. The validation and test sets underwent only resizing and normalization, ensuring that evaluation reflected real-world scenarios without artificial augmentations.

The model was trained for **50 epochs**, using a **batch size of 32**, with each epoch taking approximately **2 minutes** to complete, leading to a total training time of about **1 hour and 40 minutes**. The **AdamW optimizer** was employed, with a **learning rate of 0.001** and **weight decay of 0.0001**. Cross-Entropy Loss was minimized to optimize the classification task. Automatic mixed precision (AMP) was disabled by default to maintain full float32 training stability. The final training setup adopted for the EfficientNet model is summarized in Table 3.2.

Table 3.2: EfficientNet Training Configuration

Aspect	Details
Model Architecture	EfficientNet-B0 (pretrained on ImageNet)
Transfer Learning	Applied by replacing the classification head
Dataset	Fish4Knowledge (Fish4K-Split)
Number of Classes	23 fish species
Input Image Size	224 × 224 pixels
Loss Function	Cross-Entropy Loss
Optimizer	AdamW optimizer
Learning Rate	0.001 (fixed)
Weight Decay	0.0001
Batch Size	32 (6 dataloader workers for parallel processing)
Number of Epochs	50
Mixed Precision Training	Disabled (full float32 precision)
Data Augmentation	Random resized crop, horizontal flip, random rotation, colour jitter

Model checkpoints were systematically saved during training. After each

epoch, the model state and optimizer state were stored. The best-performing model (based on highest validation accuracy) and the last epoch model were saved separately to allow flexible evaluation and future resumption. In addition, the training could be resumed from the last checkpoint, restoring both model weights and optimizer parameters to ensure seamless continuation without loss of training dynamics.

Training and validation losses and accuracies were logged into a structured CSV file at each epoch. These logs enabled the generation of diagnostic plots illustrating the evolution of loss and accuracy across epochs. Progress bars were employed during both training and validation phases via the `tqdm` library, providing real-time feedback on training status.

Following completion of training, the best model was evaluated on the test set. Evaluation metrics including accuracy, precision, recall, F1-score, and a normalized confusion matrix were computed to comprehensively assess classification performance. All evaluation metrics and plots were saved for reporting and reproducibility purposes as can be seen in Section 4.1.2.

The training procedure was managed using a single Python script `train_efficientnet_fish4k.py`, ensuring modularity, reproducibility, and alignment with deep learning best practices.

3.2.3 Swin Transformer Model

To further extend the evaluation across architectural paradigms, the Swin Transformer model is employed. As a hierarchical vision transformer, Swin Transformer introduces a fundamentally different approach to feature extraction and classification compared to convolutional neural networks, enabling an assessment of transformer-based models in underwater scenarios.

Architecture

The Swin Transformer (Shifted Window Transformer) is a hierarchical vision transformer architecture developed to overcome the limitations of standard Vision Transformers (ViTs) in dense prediction tasks such as object detection and image segmentation. Unlike vanilla ViTs, which rely on global self-attention and fixed image resolutions, Swin Transformer introduces several architectural innovations to enable scalable, efficient, and context-aware processing of visual data [22].

At its core, the Swin Transformer applies self-attention within non-overlapping local windows of fixed size (e.g., 7×7 patches), rather than across the

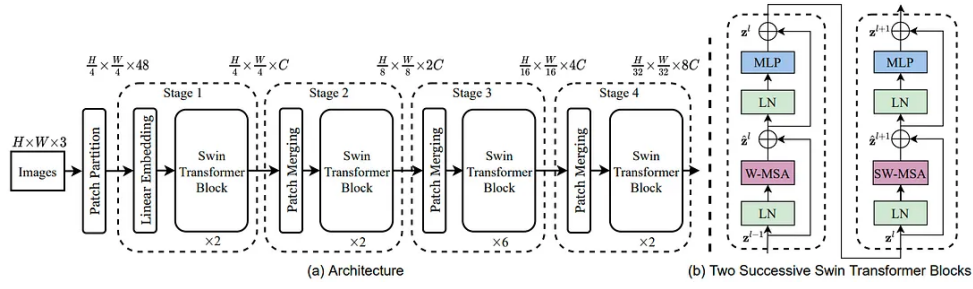


Figure 3.3: (a) The architecture of a Swin Transformer (Swin-T); (b) two successive Swin Transformer Blocks

entire image. This local self-attention mechanism drastically reduces computational complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$, where n is the number of patches.

To facilitate cross-window communication and improve global modeling capability, Swin Transformer introduces a **Shifted Window** technique. In alternating layers, the window partitioning is shifted by a fixed amount (e.g., 3 pixels), which allows neighboring patches from previous windows to attend to each other in subsequent layers. This design enables hierarchical learning while preserving computational efficiency.

Swin Transformer also constructs its architecture in a hierarchical manner, similar to CNNs as shown in Figure 3.3. This involves:

- **Patch Partitioning:** The input image is divided into fixed-size patches (e.g., 4×4), which are then linearly embedded into tokens.
- **Patch Merging:** After each stage, the number of tokens is reduced while increasing feature dimensions, mimicking downsampling in CNNs.
- **Swin Transformer Blocks:** Each stage contains multiple blocks that apply window-based multi-head self-attention (W-MSA) and shifted window-based MSA (SW-MSA).

Additionally, Swin Transformer supports both classification and dense prediction tasks by using a unified backbone. Its hierarchical structure and efficient attention mechanism make it highly effective for fine-grained recognition tasks such as underwater species classification, where both local texture and global context are important.

Training Setup and Parameters

The Swin Transformer model was trained using a transfer learning approach, initializing weights from a pretrained Swin-Tiny model trained on the ImageNet dataset [29]. Transfer learning enables faster convergence and better generalization, particularly when limited labeled data is available, by leveraging learned feature representations from a large-scale dataset. The final classification head of the Swin Transformer was modified to output 23 classes, corresponding to the distinct fish species within the Fish4Knowledge dataset.

The selection of training parameters plays a crucial role in ensuring the stability, efficiency, and performance of deep learning models. Each hyperparameter, including optimizer settings, input size, learning rate, batch size, and data augmentation strategy, has a direct impact on the model's ability to learn meaningful representations from the data without overfitting or underfitting. Therefore, careful tuning and justified selection of these values were necessary to maximize the Swin Transformer model's performance on the Fish4Knowledge dataset. The following configuration was adopted for this study, with each component selected based on its alignment with best practices for fine-tuning transformer architectures and constrained computational resources. A walk-through of training pipeline improvements that were done will be discussed later in this section.

The specific training configuration was as follows:

- **Optimizer:** The AdamW optimizer was employed due to its effectiveness in training transformer-based architectures with improved handling of weight decay. AdamW decouples weight decay from the gradient update step, leading to better generalization compared to traditional Adam, particularly when regularization is important [28].
- **Learning Rate:** An initial learning rate of 0.0001 was chosen, lower than typical CNN setups, to ensure stable and gradual fine-tuning of the pretrained Swin Transformer backbone. In contrast to a fixed learning rate approach, a cosine annealing scheduler was used to decay the learning rate toward zero over the course of training.
- **Learning Rate Scheduler:** A cosine annealing learning rate schedule was applied to gradually lower the learning rate, encouraging more refined weight updates toward the end of training and reducing the risk of overshooting minima. Cosine annealing is particularly advantageous for training Transformer-based models due to its smooth and gradual adjustment of the learning rate. Unlike traditional schedulers that implement abrupt changes, cosine annealing follows a cosine-shaped curve to reduce the learning rate progressively, which

helps in stabilizing the training process and avoiding sudden shocks to the model's learning dynamics. This smooth decay is beneficial for Transformers, which are sensitive to learning rate fluctuations, as it allows for more refined convergence towards optimal solutions. This learning rate scheduler was utilized because the early training result showed no improvements in loss and accuracy after running for a few epochs [30][31].

- **Weight Decay:** A weight decay of 0.0001 was applied to penalize large weights and mitigate overfitting. This was particularly important in fine-tuning to ensure that the Swin Transformer adapted smoothly to the smaller Fish4Knowledge dataset.
- **Batch Size:** A batch size of 32 was selected based on a balance between GPU memory limitations and ensuring statistical stability of gradient updates.
- **Number of Epochs:** The model was trained for a maximum of 50 epochs. Early stopping with a patience of 10 epochs was implemented to automatically terminate training if validation accuracy ceased to improve, optimizing both model generalization and training time.
- **Loss Function:** Cross-Entropy Loss was employed as the loss function to optimize the Swin Transformer model. This choice is standard for multi-class single-label classification problems where each image belongs to exactly one class.

Mathematically, for a given input sample with ground truth class label $y \in \{1, 2, \dots, C\}$ and model predicted probability distribution $\hat{y} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_C]$, the Cross-Entropy Loss $\mathcal{L}_{CE}$ is defined as:

$$\mathcal{L}_{CE} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

where:

- C is the total number of classes (23 fish species in this case),
- y_i is a binary indicator (0 or 1) whether class i is the correct class label,
- $\hat{y}_i$ is the model's predicted probability for class i .

In practice, the label y is provided as an integer corresponding to the correct class, and the model outputs a probability vector $\hat{y}$ after applying the softmax function to its raw logits.

This loss function measures the divergence between the true distribution (where the correct class has probability 1) and the predicted probability distribution produced by the model. Minimizing this loss encourages the model to assign higher probability to the correct class and lower probabilities to all incorrect classes, making it ideal for the fish species classification task.

Evaluation Metrics

To comprehensively assess the performance of the Swin Transformer model on the Fish4Knowledge dataset, a set of standard classification metrics was employed. These metrics evaluate the model's ability to accurately predict fish species, considering both overall accuracy and class-specific performance. The chosen metrics are described below.

Accuracy measures the proportion of correctly classified samples among the total number of samples. It is computed as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP represents true positives, TN true negatives, FP false positives, and FN false negatives. While accuracy provides a high-level view of performance, it can be misleading in imbalanced datasets, prompting the inclusion of additional evaluation metrics.

Precision quantifies the proportion of positive identifications that were actually correct. For a specific class, it is defined as:

$$\text{Precision} = \frac{TP}{TP + FP}$$

High precision reflects a low false positive rate. In this study, macro-averaged precision was computed, treating all classes equally regardless of sample size.

Recall, also referred to as sensitivity, measures the proportion of actual positives that were correctly identified. It is given by:

$$\text{Recall} = \frac{TP}{TP + FN}$$

High recall indicates that the model successfully detects most true instances. Macro-averaged recall was utilized to ensure balanced evaluation across all fish species.

The **F1-score** is the harmonic mean of precision and recall, offering a single metric that balances both false positives and false negatives:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The macro-averaged F1-score was employed to give minority classes equal importance during evaluation.

A **Normalized Confusion Matrix** was generated to visualize the Swin Transformer’s classification performance across the 23 fish species. The matrix presents the number of correct and incorrect predictions per class, highlighting patterns of confusion. Normalization was applied to account for varying sample sizes across species, enabling fairer comparison.

Additionally, the evolution of **training and validation loss and accuracy** was plotted over the course of training. These curves offer insights into model learning dynamics, convergence behavior, and potential overfitting trends.

All evaluation metrics were computed using the `scikit-learn` library, with visualizations produced using `seaborn` and `matplotlib`.

Dataset Preparation

The Fish4Knowledge dataset was originally organized for segmentation and recognition tasks, containing fish images and corresponding binary masks structured into species-specific folders. To adapt the dataset for image classification, a systematic preprocessing and reorganization process was undertaken prior to training the EfficientNet model. The same preprocessed dataset, referred to as Fish4K-Split, was reused without modification for the training and evaluation of the Swin Transformer model.

Initially, the original `fish_image/` directory, which contained subfolders named by species identifiers (e.g., `fish_01`, `fish_02`, etc.), was reorganized into a classification-compatible structure. Each subfolder exclusively contained images belonging to a single fish species, making it suitable for direct adaptation into an image classification framework.

To prepare the dataset, the images were randomly split into three subsets:

- **Training set:** 70% of the available images from each species
- **Validation set:** 20% of the available images from each species
- **Test set:** 10% of the available images from each species

The splitting was performed in a stratified manner to ensure that each subset contained images from every species, thereby preventing class omission during training or evaluation. Random shuffling was applied before splitting to eliminate ordering bias.

The final directory structure adhered to the conventions expected by PyTorch's `ImageFolder` class, as shown below:

```
Fish4K-Split/  
  train/  
    fish_01/  
    fish_02/  
    ...  
  val/  
    fish_01/  
    fish_02/  
    ...  
  test/  
    fish_01/  
    fish_02/  
    ...
```

This structure enabled seamless loading of the data using standard classification dataloaders, where each subfolder name is automatically mapped to a class label.

To verify the integrity of the dataset preparation, a small set of preview images was generated from each split and manually inspected for correct labeling. Additionally, a class distribution summary was generated and saved to a text file, recording the number of images per species across the train, validation, and test splits. This ensured reproducibility and transparency in the dataset preparation process. All preprocessing steps are documented in the `prepare_fish4k_efficientnet.py` script.

Training Procedure

Training was conducted on a local machine equipped with an AMD Ryzen 7 6800HS processor (3.20 GHz base clock) and 64 GB of RAM, running Windows 11 Pro (64-bit, version 24H2), and an Nvidia GeForce RTX 3070 Ti Laptop GPU. GPU acceleration through the dedicated graphics card was utilized to expedite training. The environment was managed via Python 3.13, using key libraries such as PyTorch, torchvision, timm, scikit-learn, NumPy, and Matplotlib. Dependency versions were managed through a

Conda environment, specified in an `environment.yml` file to ensure reproducibility.

The Swin Transformer model was trained for fish species classification following a structured and reproducible pipeline. The training utilized the Fish4K-Split dataset prepared previously, maintaining consistent training, validation, and test splits.

During training, data augmentations were dynamically applied to the training set at each epoch. These augmentations included random resized cropping and horizontal flipping, which introduced natural variability in the input images without excessive distortion. More aggressive augmentations such as random rotation and colour jitter were intentionally omitted following early experimental results, which showed that excessive augmentation degraded learning stability. The validation and test sets underwent only resizing and normalization to ensure that evaluation reflected realistic, unaltered underwater imagery.

Initially, early experiments indicated stagnation in both training and validation accuracy, with minimal reduction in loss across the first few epochs which as shown in Table C.3. Not only that, each epoch took about 4 minutes to complete, rendering a total training time of 20 minutes for 5 epochs. This prompted a series of improvements to stabilize and optimize the training process. Specifically, aggressive data augmentations such as random rotations and heavy colour jittering were omitted to avoid introducing excessive input variability. The initial learning rate was reduced from 0.001 to 0.0001 to accommodate the fine-tuning of the pretrained backbone more delicately. Automatic Mixed Precision (AMP) was integrated to speed up training and reduce memory usage without compromising model accuracy. Additionally, a cosine annealing learning rate scheduler was introduced to gradually decay the learning rate over the course of training, enabling smoother convergence. These cumulative changes led to a significant improvement in training dynamics, as reflected in consistently decreasing loss and increasing validation accuracy as summarized in Table C.4.

After the improvements have been applied, the model was trained for **50 epochs**, using a **batch size of 32**, with each epoch taking approximately **3 minutes** to complete, resulting in a total training time of about **2 hour and 30 minutes**. The **AdamW optimizer** was employed with an initial **learning rate of 0.0001** and **weight decay of 0.0001**. A **cosine annealing learning rate scheduler** was used to smoothly decay the learning rate across epochs, helping achieve a more stable convergence. Cross-Entropy Loss was minimized during training to optimize the classification objective.

Automatic Mixed Precision (AMP) was enabled to accelerate computation and reduce GPU memory usage, without adversely affecting model per-

formance. Early stopping with a patience of 10 epochs was integrated, automatically terminating training if validation accuracy did not improve for 10 consecutive epochs. The final training setup adopted for the Swin Transformer model is summarized in Table 3.3.

Table 3.3: Swin Transformer Training Configuration

Aspect	Details
Model Architecture	Swin-Tiny (pretrained on ImageNet)
Transfer Learning	Applied by replacing the classification head
Dataset	Fish4Knowledge (Fish4K-Split)
Number of Classes	23 fish species
Input Image Size	224 × 224 pixels
Loss Function	Cross-Entropy Loss
Optimizer	AdamW optimizer
Learning Rate Scheduler	Cosine Annealing Schedule
Learning Rate	0.0001 (initial)
Weight Decay	0.0001
Batch Size	32 (6 dataloader workers for parallel processing)
Number of Epochs	50 (with early stopping patience of 10 epochs)
Mixed Precision Training	Enabled (AMP)
Data Augmentation	Random resized crop, horizontal flip

Model checkpoints were systematically saved during training. After each epoch, the model state and optimizer state were stored. The best-performing model (based on highest validation accuracy) and the last epoch model were saved separately to allow flexible evaluation and future resumption. In addition, training could be resumed from the last checkpoint, restoring both model weights and optimizer parameters to ensure seamless continuation without loss of training dynamics.

Training and validation losses, accuracies, and learning rates were logged into a structured CSV file at each epoch. These logs enabled the generation of diagnostic plots illustrating the evolution of loss, accuracy, and learning rate across epochs. Progress bars were employed during both training and validation phases via the `tqdm` library, providing real-time feedback on training status.

Following completion of training, the best model was evaluated on the test set. Evaluation metrics including accuracy, precision, recall, F1-score, and a normalized confusion matrix were computed to comprehensively assess classification performance. All evaluation metrics and plots were saved for reporting and reproducibility purposes as can be seen in Section ??.

The training procedure was managed using a single Python script `train_swin_fish4k.py`, ensuring modularity, reproducibility, and alignment with deep learning best practices.

3.3 Conclusion

This chapter described the systematic methodology employed throughout the study, from data selection and preprocessing to model selection, training, and evaluation. The Fish4Knowledge dataset was adapted to meet the requirements of both classification and detection tasks. YOLOv8, EfficientNet, and Swin Transformer models were selected for implementation based on their proven effectiveness in image-based tasks. Each model's training process, parameter choices, and evaluation criteria were carefully defined to ensure a fair and comprehensive comparison. The next chapter presents the experimental results and analyzes the performance of the trained models on the Fish4Knowledge dataset.

4 Result and Evaluation

This chapter presents the experimental results and evaluation of the deep learning models developed for underwater fish species detection and classification. The models were assessed using both quantitative metrics and qualitative visualizations to comprehensively measure their effectiveness. Comparative analysis between the models is also conducted to highlight their respective strengths, weaknesses, and suitability for real-world deployment in underwater environments.

4.1 Evaluation of Model Performance

This section presents a comprehensive evaluation of the three deep learning models developed in this study—YOLOv8, EfficientNet, and Swin Transformer—on the Fish4Knowledge dataset. Each model's performance is assessed using appropriate quantitative metrics, such as accuracy, precision, recall, F1-score, and mean Average Precision (mAP), depending on the task.

For classification models (EfficientNet and Swin Transformer), performance is primarily evaluated through accuracy, macro-averaged precision, recall, F1-score, and normalized confusion matrices. For the object detection model (YOLOv8), evaluation is conducted using standard detection metrics, including precision, recall, F1-score, mAP@0.5, and mAP@0.5:0.95, in addition to qualitative visualizations of predicted bounding boxes.

Diagnostic plots illustrating the evolution of training and validation loss and accuracy across epochs are provided for each model, offering insights into their learning behaviors, convergence rates, and potential signs of overfitting or underfitting. Qualitative visualizations complement the quantitative evaluation by presenting examples of correct and incorrect model predictions.

Through this systematic evaluation, the effectiveness of each model is critically analyzed, paving the way for a comparative performance analysis and deeper discussion of architectural strengths and trade-offs in subsequent sections.

4.1.1 YOLOv8 Model

Following the training of the YOLOv8 object detection model on the Fish4Knowledge dataset, several diagnostic plots were generated to assess its predictive performance. These figures provide detailed insights into the model's ability to detect and classify 23 distinct underwater fish species under realistic aquatic conditions.

In addition to quantitative evaluation through precision, recall, mean Average Precision (mAP), and loss curves, qualitative visualizations were also produced. These visualizations compare ground truth annotations with the model's predictions on a batch of test images. By examining both statistical metrics and visual outputs, a comprehensive understanding of the model's detection accuracy, classification reliability, and generalization ability was obtained.

Training Curves and Metric Evolution

The training process of the YOLOv8 model was closely monitored through a series of diagnostic plots generated over the course of 35 epochs. These plots illustrate the evolution of key metrics and provide insights into the model's learning behaviour.

The first plot illustrates the **learning rates over epochs** as can be seen in Figure B.6. This plot displays the learning rate schedule applied during training. The learning rate initially increases sharply at a shy below 0.01 during the first few epochs (known as warm-up) to stabilize model updates. After reaching the peak, it gradually decreases over the remaining epochs to fine-tune the model weights carefully until it reaches about 0.005, where the training stops. This strategy helps prevent unstable training in early stages and overfitting in later stages, supporting both faster convergence and improved final performance.

The second plot shown is the **training losses over epochs** in Figure B.7. As mentioned previously, YOLOv8 uses a combination of 3 loss functions namely, box loss, class loss, and DFL loss. The training loss curves show the evolution of box loss, class loss, and distribution focal loss (DFL) on the training set. All loss components demonstrate a smooth and continuous decrease throughout training. Class loss, in particular, drops significantly from 2.25 to 0.5 during the first 10 epochs, reflecting rapid learning of fish species classification. While box loss and DFL started out at 0.94 and 1.23, they both went down and reach a stable value of about 0.49 and 0.96, respectively. The consistent downward trend in training losses indicates effective optimization and absence of major issues like training instability or

vanishing gradients.

The next plot displays the **validation losses over epochs** in Figure B.8. The validation losses plot tracks three components: box loss, class loss, and distribution focal loss (DFL) on the validation set. Box loss reflects localization accuracy, class loss reflects classification accuracy, and DFL sharpens bounding box regression. All three losses show a declining trend over epochs, with class loss reducing most sharply during early training from 0.73 down to 0.22 by 20th epoch. As for the box loss and, it began with a value of 0.67, then it spiked for one epoch to 0.72, which then gradually declined until 0.29 by 50th epoch. In regards to DFL loss, it too spiked at the beginning which then gradually decrease throughout the training process. The flattening of the curves after approximately 25 epochs indicates that the model reached a stage of diminishing returns, suggesting convergence toward optimal validation performance.

Moving on to the next one is the **precision and recall over epochs** in Figure B.9. This plot shows how precision and recall scores evolved throughout training. Precision measures the proportion of correct positive predictions, while recall measures the proportion of actual positives correctly identified. Initially, recall was considerably lower than precision, indicating that the model was missing many detections. Over time, both precision and recall improved steadily, converging toward high values above 0.95 by epoch 30. This suggests that the model progressively learned to detect more fish instances while maintaining accuracy in its predictions.

Lastly is the **mean average precision (mAP) over epochs** which can be referred to in Figure B.10. This plot displays the growth of mAP@0.5 and mAP@0.5:0.95 scores over epochs. Mean Average Precision (mAP) measures detection performance by evaluating both localization and classification quality. mAP@0.5 represents correct detections with at least 50% overlap, while mAP@0.5:0.95 is a stricter metric averaging performance across thresholds. Both curves show steady improvement, with mAP@0.5 exceeding 0.98 and mAP@0.5:0.95 approaching 0.93 by the end of training. This demonstrates that the model was able to make highly accurate and consistently precise detections as training progressed.

In the next following diagrams are diagnostic plots that were generated by the built-in evaluation function of YOLOv8. These figures offer a comprehensive view of the model's performance in detecting and classifying 23 different underwater fish species in realistic aquatic environments.

Confusion Matrix

Figure B.3 presents the **absolute confusion matrix**, which provides a comprehensive evaluation of the YOLOv8 model's classification performance across 23 fish species and one background class. The matrix is strongly diagonal-dominant, indicating a high level of classification accuracy. Several species exhibit near-perfect predictions, including *Chromis chrysura*, *Amphiprion clarkii*, *Chaetodon lunulatus*, *Chaetodon trifascialis*, *Myripristis kuntee*, and *Hemigymnus fasciatus*, all of which had zero off-diagonal misclassifications. The species *Dascyllus reticulatus*, which had the largest number of instances, showed high classification fidelity with only minor misclassifications into visually similar classes and a small number of predictions labeled as background.

A few species, such as *Acanthurus nigrofuscus*, *Canthigaster valentini*, and *Neoglyphidodon nigroris*, exhibited mild confusion with other classes, likely due to visual similarities in morphology or environmental lighting. For instance, *Acanthurus nigrofuscus* had isolated misclassifications into related species, yet maintained the majority of predictions correctly. These limited off-diagonal entries do not indicate systemic error, but rather natural class overlap or limited sample size.

The background class demonstrated minimal false positives, suggesting effective suppression of spurious detections. Overall, the model demonstrated excellent generalization and class separability. To further reduce residual confusion, enhancements such as data augmentation, anchor optimization, or improved training balance for low-frequency classes may be considered. This matrix confirms that the YOLOv8 model is highly reliable for multi-class underwater species detection, even in the presence of fine-grained inter-class similarities.

Figure 4.1 further shows the **normalized confusion matrix**, where each row is scaled from 0 to 1. This reveals that most species are predicted with **100% accuracy**, and only a few classes exhibit minor confusion (e.g., *Acanthurus nigrofuscus*, which shows 73% correct predictions and some mislabeling). The matrix suggests **high inter-class separability**, a strong indicator of reliable species identification.

The normalized confusion matrix further illustrates the class-wise recall performance of the YOLOv8 model across 23 underwater fish species and one background class, where each row is scaled from 0 to 1. The matrix exhibits strong diagonal dominance, indicating that the majority of species are predicted with high reliability. A significant number of species, including *Chromis chrysura*, *Amphiprion clarkii*, *Chaetodon lunulatus*, *Myripristis kuntee*, *Hemigymnus fasciatus*, *Scaridae*, and *Balistapus un-*

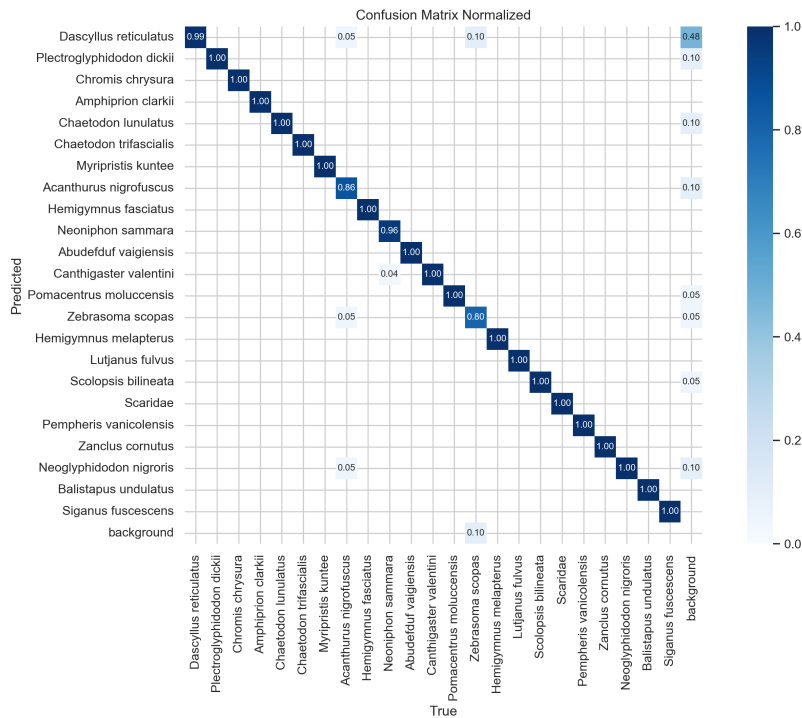


Figure 4.1: YOLOv8 Normalized Confusion Matrix

dulatus, achieved perfect recall values of 1.00, with no misclassifications recorded. Several other species, such as *Dascyllus reticulatus* and *Neoniphon sammara*, maintained near-perfect recall with values of 0.99 and 0.96, respectively, accompanied by minor misclassification into similar classes or background.

A small subset of classes demonstrated moderate confusion, notably *Acanthurus nigrofasciatus* and *Zebrasoma scopas*, which had recall values of 0.86 and 0.80. These misclassifications may be attributed to overlapping visual characteristics or class imbalance. Additionally, some species such as *Acanthurus nigrofasciatus* and *Neoglyphidodon nigroris* showed limited but notable confusion with the background class, each exhibiting 0.10 misclassification into background. Despite these exceptions, most off-diagonal values were zero, reflecting strong class separability.

Overall, the normalized matrix confirms that the model is highly reliable for multi-class fish detection, with excellent recall performance and minimal inter-class confusion, making it well-suited for practical underwater species recognition tasks.

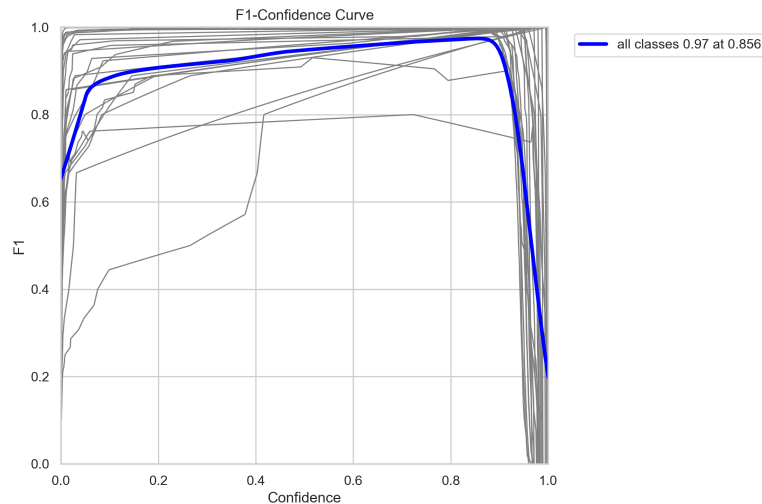


Figure 4.2: YOLOv8 F1-Confidence Curve

F1-Confidence, Precision-Confidence, Recall-Confidence, and Precision-Recall (PR) Curves

Figure 4.2 illustrates the **F1-confidence curve**, which evaluates the model's balance between precision and recall across different confidence thresholds. The model achieved a peak **F1 score of 0.97** at a threshold of **0.856**, indicating excellent harmony between how often it predicts correctly (precision) and how often it detects all relevant objects (recall).

Figure B.4 and B.5 show how precision and recall vary with model confidence. The **Precision-Confidence curve** reveals a high and stable precision, peaking at **1.00**, meaning nearly all predictions are correct at high confidence levels. While the **Recall-Confidence curve** shows recall approaching **0.99** when confidence thresholds are low, indicating the model successfully detects almost all fish when permitted more flexibility. Together, these curves affirm that the model is not overly conservative or aggressive in its detections, and performs robustly across varying thresholds.

Figure 4.3 displays the **Precision-Recall curve**, a key performance measure in object detection. The model achieves a mean average precision of **0.982 at IoU 0.5 (mAP@50)**, suggesting that it accurately localizes and classifies fish species with near-perfect confidence. The curve hugs the top-right corner, which is characteristic of high-performance detectors.

In addition to quantitative metrics, visual inspection of predictions provides further insight into the model's effectiveness. Sample images comparing ground truth and predicted outputs are shown below.

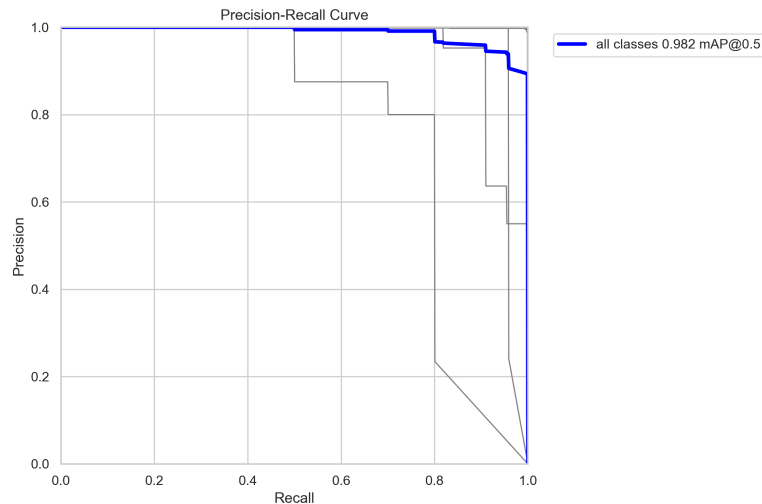


Figure 4.3: YOLOv8 Precision-Recall Curve

Qualitative Visualization of Ground Truth and Predictions

To further assess the YOLOv8 model's performance beyond numerical metrics, qualitative visualizations were generated by comparing ground truth annotations against model predictions on the test set. Figures B.12 and B.11 illustrate a direct comparison between the expected outputs and the actual outputs produced by the trained model.

Figure B.12 displays a subset of test images annotated with ground truth bounding boxes and species labels. Each bounding box precisely encloses a fish instance and is labeled with its corresponding species name. The ground truth annotations provide a clear reference for the spatial localization and classification that the model was expected to learn during training.

Figure B.11 shows the corresponding model predictions for the same set of test images. Each bounding box indicates the model's predicted location and species label for detected fish instances. Additionally, a confidence score is displayed next to each prediction, representing the model's estimated certainty for the predicted class.

Overall, the model predictions closely match the ground truth annotations, both in terms of bounding box positioning and species classification. The bounding boxes accurately encapsulate the fish instances, and the species names largely align with the expected labels. Confidence scores are consistently high (ranging from 0.9 to 1.0), suggesting that the model makes predictions with strong certainty. Minor variations in box placements are observed but remain within acceptable limits, indicating reliable spatial detection. These qualitative results corroborate the high precision, recall,

and mAP values obtained through quantitative evaluation.

Conclusion

The performance of the YOLOv8 model trained on the Fish4Knowledge dataset was evaluated using standard object detection metrics and qualitative visualizations. Table 4.1 summarizes the key results obtained after 35 epochs of training, including precision, recall, mean Average Precision (mAP), and F1 score.

Table 4.1: Summary of YOLOv8 Model Performance on Fish4Knowledge Dataset

Metric			Value
Mean	Average	Precision	0.982
(mAP@0.5)			
Mean	Average	Precision	0.933
(mAP@0.5:0.95)			
Precision			0.973
Recall			0.978
F1 Score (at confidence threshold 0.856)			0.970
Number of Training Epochs			50 epochs
Average Training Time per Epoch			~4 minutes
Total Training Time			~3 hours

The strong performance of the YOLOv8 model—evidenced by high mAP@0.5 (0.982), F1 score (0.97), and a largely accurate confusion matrix—can be attributed in part to its default composite loss function. This loss function optimizes bounding box localization (via CIoU), classification accuracy (via BCE), and coordinate precision (via DFL), allowing the model to learn effectively from the Fish4Knowledge dataset without manual reconfiguration.

The model evaluation metrics collectively confirm that the YOLOv8 model is:

1. Highly accurate across all fish species
2. Balanced in prediction confidence and generalization
3. Well-suited for real-world underwater detection tasks

These results validate the overall training approach and demonstrate that

the preprocessed Fish4Knowledge dataset provided high-quality annotations suitable for deep learning-based object detection.

4.1.2 EfficientNet Model

Following the training of the EfficientNet classification model on the Fish4Knowledge dataset, several diagnostic plots were generated to assess its predictive performance. These figures provide detailed insights into the model's ability to detect and classify 23 distinct underwater fish species under realistic aquatic conditions.

Training Curves and Metric Evolution

The training process of the EfficientNet model was closely monitored through a series of diagnostic plots generated over the course of 50 epochs. These plots illustrate the evolution of key metrics and provide insights into the model's learning behaviour.

The evolution of training and validation accuracy and loss across epochs is depicted in Figure 4.4 and Figure 4.5, respectively.

The training accuracy showed a rapid improvement during the initial epochs, reaching approximately **94%** within the first 10 epochs. Subsequently, the accuracy curve continued to rise steadily, albeit at a slower rate, plateauing around **97.7%** by the 50th epoch. Validation accuracy, meanwhile, exhibited consistently higher performance than training accuracy throughout the training process, peaking at approximately **99.2%** validation accuracy at the end of training.

The loss curves demonstrate a corresponding decline in training and validation loss. Training loss exhibited a sharp decrease during early epochs and gradually plateaued, while validation loss maintained a consistently lower value throughout, indicating good generalization without evidence of overfitting. Importantly, no significant divergence was observed between training and validation loss, suggesting stable optimization and effective transfer learning.

The consistent gap between training and validation metrics, with validation performing better, can be attributed to the relatively simple classification problem enabled by pre-trained EfficientNet feature representations and robust data augmentations.

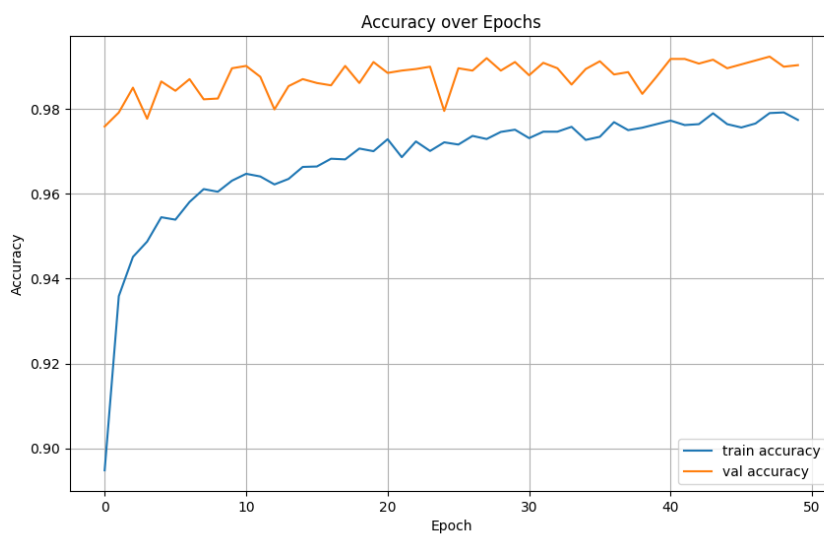


Figure 4.4: EfficientNet Training and Validation Accuracy over Epochs

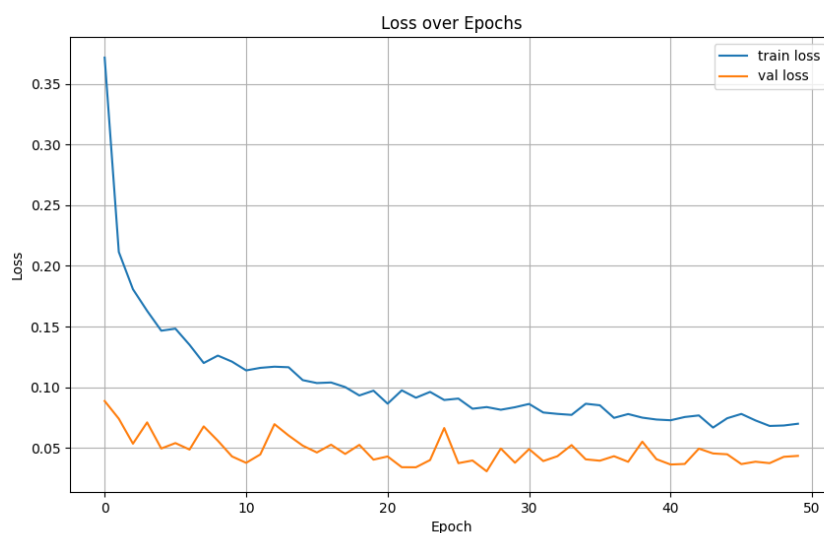


Figure 4.5: EfficientNet Training and Validation Loss over Epochs

F1-Score, Precision, Recall, and Accuracy

Final model performance on the test set was evaluated using accuracy, as well as macro-averaged precision, recall, and F1-score. Macro-averaging treats all classes equally regardless of class imbalance, ensuring that minority classes are not overshadowed by majority classes during evaluation.

The model achieved an impressive **precision of 98.54%**, indicating that the vast majority of predicted species labels were correct. **Recall reached 94.82%**, suggesting that most true fish species present in the test set were successfully identified by the model. The resulting **F1-score**, which balances precision and recall, was **96.16%**. The overall accuracy that the model achieved was spectacular at **99.09%**.

These results affirm that the model maintained a strong balance between minimizing false positives and false negatives across the 23-class classification problem, validating both the effectiveness of transfer learning and the suitability of the dataset preparation and augmentation strategies.

Confusion Matrix

The normalized confusion matrix for the EfficientNet model's predictions on the test set is presented in Figure 4.6 where each row is scaled from 0 to 1. To enhance clarity, only nonzero prediction values are displayed, omitting clutter from zero entries.

Overall, the model demonstrated **excellent species-level classification accuracy**, with most classes achieving near-perfect or perfect normalized scores along the diagonal of the matrix. Most species, including *Dascyllus reticulatus*, *Amphiprion clarkii*, and *Myripristis kuntee*, were consistently classified with very high precision, reflecting the model's strong discriminative ability.

However, a few minor misclassifications were observed among some visually similar species. Notably, slight confusion occurred for classes such as *Abudefduf vaigiensis*, and *Scarus* (Scaridae), where the model occasionally predicted neighboring classes. A higher confusion occurred on the *Neoglyphidodon nigroris*, where there is a 50/50 chance of the model predicting it correctly or the visually indistinguishable *Dascyllus reticulatus*. These errors are likely attributable to natural visual similarity between species or limited representation within the training dataset.

Despite these rare misclassifications, the confusion matrix remains strongly diagonal-dominant, confirming that the EfficientNet model achieved **robust**

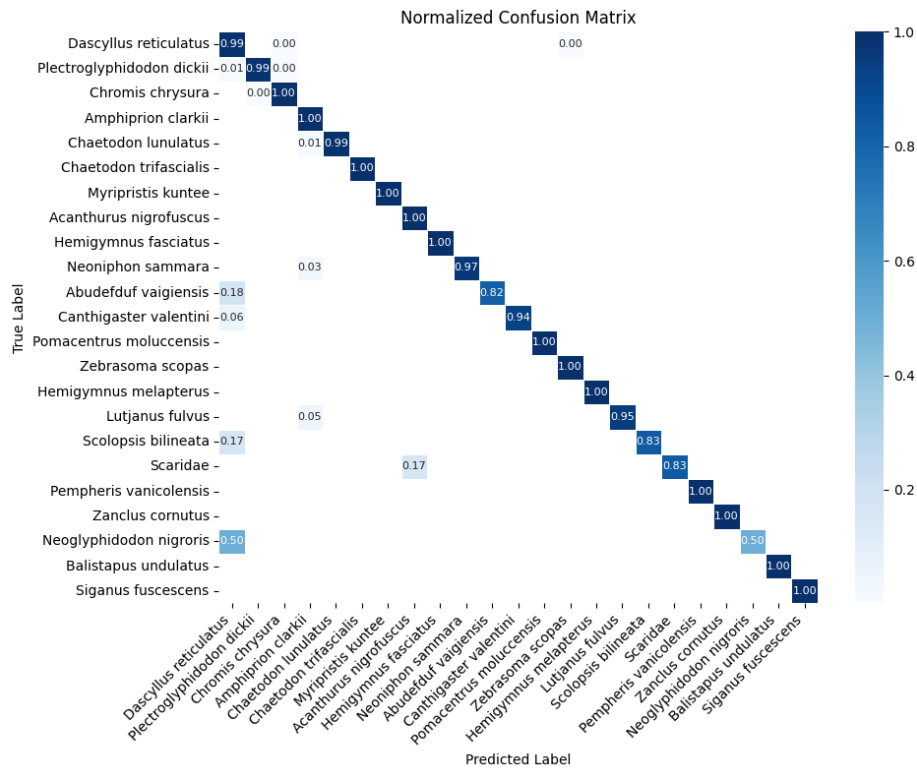


Figure 4.6: EfficientNet Normalized Confusion Matrix

and reliable fine-grained classification performance on the Fish4Knowledge test set. The high confidence and consistency observed across most species reinforce the effectiveness of the transfer learning and training strategies employed.

Qualitative Visualization of Predictions

To further assess the EfficientNet model's performance beyond numerical metrics, qualitative visualizations were generated by comparing predicted species labels against the ground truth labels on a randomly selected subset of test images. Figures B.13 and B.14 illustrate a direct comparison between the model's expected outputs and the actual outputs produced after training.

Each figure displays a 4x4 grid of test images, where each image is annotated with both the model's predicted species and the true species label. Correct predictions are shown with green-coloured titles, while incorrect predictions are highlighted in red. This colour-coding provides a clear and immediate visual assessment of the model's classification accuracy across different fish species.

In Figure B.13, the model successfully classified the majority of fish images, correctly identifying species such as *Amphiprion clarkii*, *Dascyllus reticulatus*, *Plectroglyphidodon dickii*, and *Chromis chrysura*. These correct predictions demonstrate the model’s ability to reliably distinguish between key underwater species even in varied visual conditions.

Likewise, Figure B.14 demonstrates flawless classification performance, with all images correctly classified by the model. The consistent green-coloured titles across all images indicate that the model achieved perfect agreement with the ground truth labels for this subset, further affirming its generalization ability and robustness.

Overall, the qualitative visualizations corroborate the high test set precision (98.54%), F1-score (96.16%), and accuracy (99.09%) obtained during quantitative evaluation. The majority of predictions were correct with high visual confidence, while misclassifications were rare and largely confined to biologically similar species classes. These results further validate the effectiveness of using EfficientNet for underwater species classification tasks.

Conclusion

The performance of the EfficientNet model trained on the Fish4Knowledge dataset was evaluated using standard multi-class classification metrics and qualitative visualizations. Table 4.2 summarizes the key results obtained after 50 epochs of training, including precision, recall, F1 score, and accuracy.

Table 4.2: Summary of EfficientNet Model Performance on Fish4Knowledge Dataset

Metric	Value
Precision	0.9854
Recall	0.9482
F1 Score	0.9616
Best Validation Accuracy	0.9923
Final Test Accuracy	0.9909
Number of Training Epochs	50 epochs
Average Training Time per Epoch	~2 minutes
Total Training Time	~1 hour 40 minutes

The strong classification performance of the EfficientNet model—evidenced by the high final test accuracy (0.9909), F1 score (0.9616), and the largely accurate and diagonally dominant confusion matrix—can be attributed to

the transfer learning strategy adopted. Leveraging pretrained weights from ImageNet allowed the model to converge quickly and generalize effectively to the Fish4Knowledge dataset, even with a moderate training time.

The model evaluation metrics collectively confirm that the EfficientNet model is:

1. Highly accurate across all 23 fish species
2. Balanced in minimizing both false positives and false negatives
3. Well-suited for real-world underwater classification tasks

These results validate the overall training approach and demonstrate that the preprocessed Fish4Knowledge dataset provided high-quality class labels suitable for deep learning-based underwater species classification.

4.1.3 Swin Transformer Model

Training Curves and Metric Evolution

The training process of the Swin Transformer model was systematically monitored through a series of diagnostic plots generated over the course of 50 epochs. These plots illustrate the evolution of key metrics and provide insights into the model's learning behavior and convergence dynamics.

The evolution of training and validation accuracy and loss across epochs is depicted in Figure 4.7 and Figure 4.8, respectively. Additionally, the learning rate evolution following the cosine annealing schedule is shown in Figure B.15.

The training accuracy exhibited rapid growth during the early epochs, reaching approximately **97%** within the first 10 epochs. Thereafter, it continued to improve steadily, plateauing around **99.0%** training accuracy by epoch 50. Validation accuracy, meanwhile, remained consistently high across the training process, exceeding **99.5%** from an early stage and maintaining near-perfect levels through to the end of training.

The loss curves reflect a strong and stable learning trajectory. Training loss declined sharply during the initial epochs and gradually continued to decrease, while validation loss remained consistently low with minimal fluctuations. No signs of significant overfitting were observed, as the validation loss closely tracked the behavior of training loss throughout training.

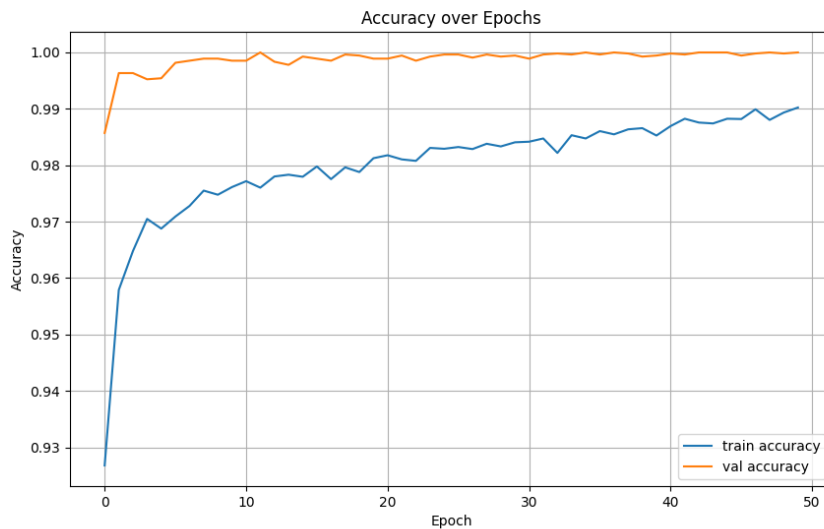


Figure 4.7: Swin Transformer Training and Validation Accuracy over Epochs

The learning rate plot illustrates the smooth decay of the learning rate from the initial value of 0.0001 down to approximately 0.00003, following a cosine annealing schedule. This learning rate strategy helped stabilize the model's convergence and contributed to its strong final performance.

Overall, the training and validation curves confirm that the Swin Transformer model successfully learned robust feature representations for fish species classification, with stable optimization and minimal evidence of instability or overfitting.

F1-Score, Precision, Recall, and Accuracy

Final model performance on the test set was evaluated using accuracy, as well as macro-averaged precision, recall, and F1-score. Macro-averaging treats all classes equally regardless of class imbalance, ensuring that minority classes are not overshadowed by majority classes during evaluation.

The Swin Transformer model achieved a remarkable **precision of 98.67%**, demonstrating that the vast majority of predicted species labels were accurate. **Recall reached 97.41%**, indicating that most true fish species present in the test set were successfully identified by the model. The resulting **F1-score**, which balances precision and recall, was **97.60%**. Ultimately, the overall accuracy that the model achieved was an outstanding **99.71%**.

These results confirm that the model maintained a strong balance between minimizing false positives and false negatives across the 23-class fish

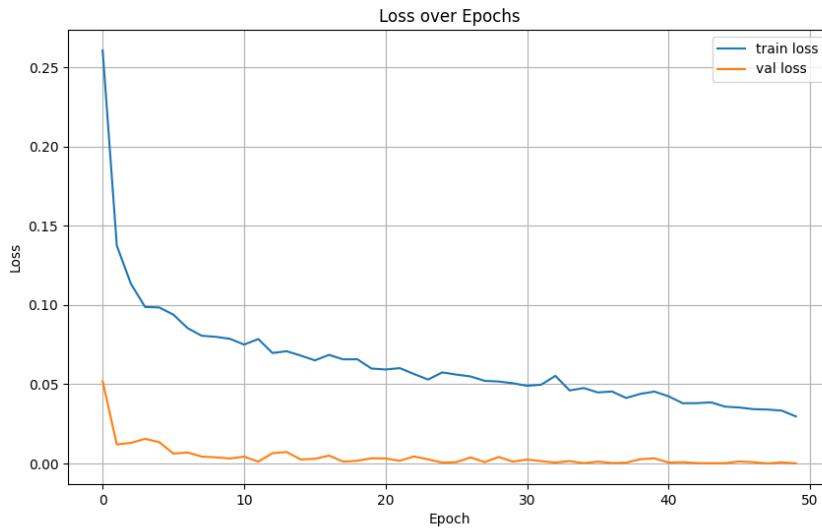


Figure 4.8: Swin Transformer Training and Validation Loss over Epochs

species classification task. The performance highlights both the effectiveness of transfer learning using the Swin Transformer architecture and the robustness of the dataset preparation and augmentation strategies.

Confusion Matrix

The normalized confusion matrix for the Swin Transformer model's predictions on the test set is presented in Figure 4.9, where each row is scaled from 0 to 1. To enhance clarity, only nonzero prediction values are displayed, omitting clutter from zero entries.

Overall, the model demonstrated **outstanding species-level classification accuracy**, with most classes achieving near-perfect or perfect normalized scores along the diagonal of the matrix. Species such as *Dascyllus reticulatus*, *Plectroglyphidodon dickii*, and *Chromis Chrysura* were consistently classified with exceptionally high precision, showcasing the model's strong discriminative power.

Minor misclassifications were observed in a few cases. Specifically, slight confusion occurred for *Acanthurus nigrofuscus*, where a small fraction of instances were misclassified as *Scaridae*, and a higher level of confusion was noted for *Neoglyphidodon nigroris*, where a 50% prediction overlap with *Dascyllus reticulatus* was evident. These misclassifications likely stem from the high morphological similarity between these species in the underwater environment and possible limited sample representation in the training set.

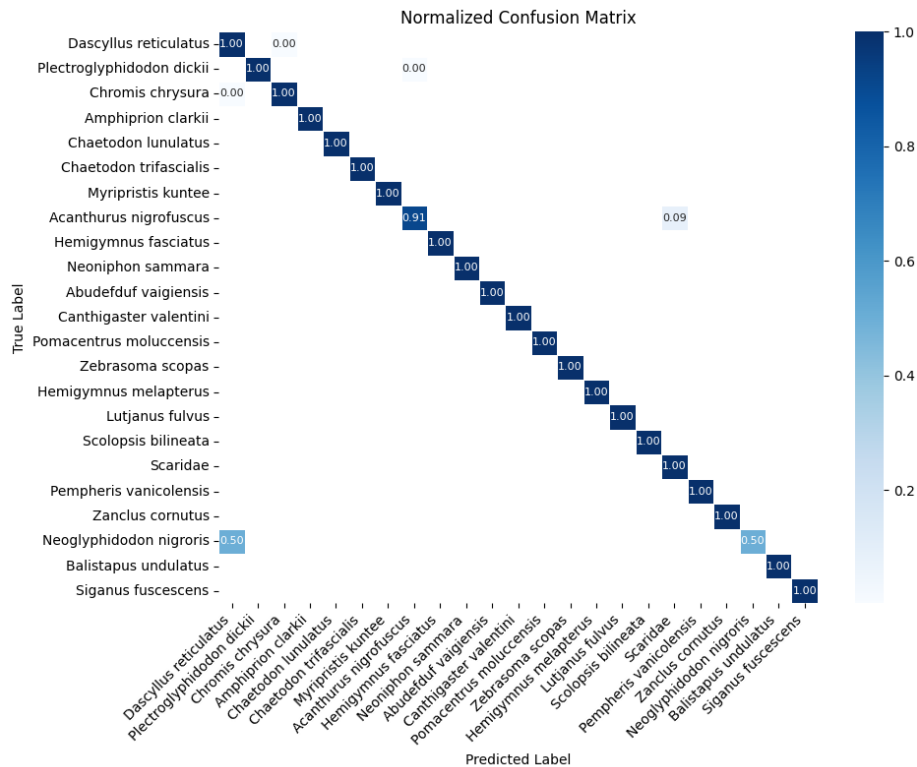


Figure 4.9: Swin Transformer Normalized Confusion Matrix

Despite these isolated errors, the confusion matrix remains strongly diagonal-dominant, affirming that the Swin Transformer model achieved **robust and reliable fine-grained classification** performance on the Fish4Knowledge test set. The consistent high-confidence predictions across most species reinforce the effectiveness of the training pipeline, learning rate scheduling, and transfer learning strategies applied.

Qualitative Visualization of Predictions

To further assess the Swin Transformer model's performance beyond numerical metrics, qualitative visualizations were generated by comparing predicted species labels against the ground truth labels on a randomly selected subset of test images. Figures B.16 and B.17 illustrate a direct comparison between the model's expected outputs and the actual outputs produced after training.

Each figure displays a 4x4 grid of test images, where each image is annotated with both the model's predicted species and the true species label. Correct predictions are shown with green-coloured titles, while incorrect predictions are highlighted in red. This colour-coding provides a clear and

immediate visual assessment of the model's classification accuracy across different fish species.

In Figure B.16, the model successfully classified the majority of fish images, correctly identifying species such as *Dascyllus reticulatus*, *Chromis chrysura*, *Plectroglyphidodon dickii*, and *Amphiprion clarkii*. These correct predictions demonstrate the model's ability to reliably distinguish between different underwater species even under visually challenging conditions.

Similarly, Figure B.17 shows consistently accurate performance. The overwhelming majority of predictions were correct, as indicated by the dominance of green-coloured titles, suggesting that the model generalizes effectively across different samples and species variations.

Overall, the qualitative visualizations corroborate the high test set precision (98.67%), F1-score (97.60%), and accuracy (99.71%) obtained during quantitative evaluation. The model demonstrated strong classification reliability with minimal confusion, further validating the success of the transfer learning approach and the robustness of the training strategy.

Conclusion

The performance of the Swin Transformer model trained on the Fish4Knowledge dataset was evaluated using standard multi-class classification metrics and qualitative visualizations. Table 4.3 summarizes the key results obtained after 50 epochs of training, including precision, recall, F1 score, and accuracy.

Table 4.3: Summary of Swin Transformer Model Performance on Fish4Knowledge Dataset

Metric	Value
Precision	0.9867
Recall	0.9741
F1 Score	0.9760
Best Validation Accuracy	1.0
Final Test Accuracy	0.9971
Number of Training Epochs	50 epochs
Average Training Time per Epoch	~3 minutes
Total Training Time	~2 hours 30 minutes

The strong classification performance of the Swin Transformer model—evidenced by the high final test accuracy (0.9971), F1 score (0.9760), and the largely accurate and diagonally dominant confusion matrix—can be attributed

to the use of a hierarchical Vision Transformer backbone pre-trained on ImageNet. Transfer learning allowed the model to leverage powerful multi-scale feature representations, leading to rapid convergence and excellent generalization on the Fish4Knowledge dataset despite a moderate total training time.

The model evaluation metrics collectively confirm that the Swin Transformer model is:

1. Highly accurate across all 23 fish species
2. Balanced in minimizing both false positives and false negatives
3. Well-suited for real-world underwater classification tasks

These results validate the overall training approach and demonstrate that the preprocessed Fish4Knowledge dataset provided high-quality annotations suitable for deep learning-based underwater species classification using Transformer-based models.

4.2 Comparative Analysis of Model Performance

This section presents a comparative analysis of the three deep learning models developed and evaluated in this study—YOLOv8, EfficientNet, and Swin Transformer—based on their performance on the Fish4Knowledge dataset. The models are compared across key quantitative metrics, including precision, recall, F1-score, final test accuracy, mean Average Precision (mAP), and model size.

Through a systematic comparison, insights are drawn into the trade-offs between model complexity, classification or detection performance, and real-world applicability. The architectural characteristics influencing model performance are also examined to better understand the strengths and limitations of each approach within the context of underwater species detection and classification tasks.

Discussion on Performance Difference

The comparative performance of YOLOv8, EfficientNet, and Swin Transformer models was evaluated based on precision, recall, F1-score, final

test accuracy (for classifiers), mean Average Precision (for detectors), and model size. Table C.5 summarizes the key results obtained from each model.

Across all models, strong performance was observed; however, distinct differences emerged:

- **Precision:** Swin Transformer achieved the highest precision (0.987), marginally outperforming EfficientNet (0.985) and YOLOv8 (0.970). This reflects Swin Transformer's ability to minimize false positive predictions.
- **Recall:** Swin Transformer again led with a recall of 0.974, followed by YOLOv8 (0.971) and EfficientNet (0.948). The superior recall of Swin Transformer suggests better sensitivity in detecting correct fish species across the test set.
- **F1-Score:** The Swin Transformer attained the highest F1-score (0.976), indicating the best overall balance between precision and recall among the evaluated models.
- **Final Test Accuracy and mAP:** For classification models, Swin Transformer achieved a final test accuracy of 99.70%, exceeding EfficientNet's already strong 99.10%. For YOLOv8, the mAP@0.5 was an impressive 0.982, with mAP@0.5:0.95 at 0.933, confirming its high object detection accuracy in challenging underwater environments.
- **Model Size:** EfficientNet-B0 provided an optimal trade-off between performance and model size, occupying only ~50 MB while achieving excellent precision and recall. In contrast, Swin Transformer, despite offering the best performance, required significantly more storage at ~323 MB. YOLOv8n, designed for lightweight real-time detection, maintained a very compact model size of ~12 MB.

Overall, while Swin Transformer outperformed in almost all evaluation metrics, EfficientNet offered competitive performance with a significantly smaller model footprint. YOLOv8 demonstrated exceptional detection capabilities, maintaining high precision and recall while achieving real-time applicability due to its compact size.

These results illustrate the trade-offs between model size, speed, and classification/detection performance when deploying deep learning models in real-world underwater scenarios.

Analysis of Model Architecture and Impact

The architectural design of each model had a significant influence on its performance in underwater fish species detection and classification tasks. Understanding the structural differences between YOLOv8, EfficientNet, and Swin Transformer helps explain the observed variations in precision, recall, F1-score, and computational footprint.

YOLOv8n is a one-stage object detector optimized for real-time applications. Its design integrates spatial pyramid pooling and efficient convolutional operations, enabling rapid detection with high precision even on lightweight hardware. Despite its smaller model size (~ 12 MB), YOLOv8 achieved strong detection performance, with mAP@0.5 reaching 0.982, albeit at a longer training time of approximately 3 hours. However, its slightly lower recall compared to Swin Transformer suggests a minor trade-off in detecting all true positive instances, particularly smaller or camouflaged fish species in cluttered underwater scenes.

EfficientNet-B0 adopts a compound scaling approach, systematically balancing network depth, width, and resolution. Its lightweight design (~ 50 MB) provided an excellent balance between computational efficiency – training time of just shy below the 2 hour mark – and classification performance. EfficientNet achieved a final test accuracy of 99.09%, demonstrating its capability to generalize well despite limited underwater-specific features in the dataset. However, compared to the Swin Transformer, its slightly lower recall suggests that EfficientNet may have been more conservative, favoring precision over sensitivity.

Swin Transformer Tiny leverages a hierarchical Transformer architecture with shifted window-based self-attention mechanisms. This design enables both local and global feature extraction, making it highly effective in complex visual environments like underwater imagery. Although the model size is substantially larger (~ 323 MB) and longer training time (2.5 hours), the Swin Transformer achieved the highest overall performance across all metrics, including a final test accuracy of 99.71% and an F1-score of 0.9760. Its ability to simultaneously model fine-grained textures and broader spatial context likely contributed to superior detection of varied fish species across challenging underwater conditions.

Overall, the comparative results highlight a clear trade-off between model complexity, storage requirements, and predictive performance. While Swin Transformer offers the best overall accuracy and robustness, EfficientNet provides a competitive alternative with significantly lower computational demands. YOLOv8, despite being a detection-focused model, achieved remarkable results with minimal resource usage, making it suitable for real-

time deployment scenarios where both speed and accuracy are critical.

Conclusion

The comparative evaluation of YOLOv8, EfficientNet, and Swin Transformer models highlights important trade-offs between architectural complexity, predictive performance, and computational efficiency for underwater species detection and classification.

Among the three models, the Swin Transformer demonstrated the highest overall performance across all classification metrics. Its hierarchical Transformer architecture proved highly effective at capturing both fine-grained and contextual features in challenging underwater imagery. However, this performance came at the cost of significantly larger model size and longer training time when compared to EfficientNet.

EfficientNet-B0 offered a competitive alternative, achieving strong performance while maintaining a much smaller model size, and the fastest training time among all models. This balance between efficiency and accuracy makes EfficientNet a practical choice for scenarios where computational resources are limited.

YOLOv8, although primarily an object detector rather than a pure classifier, achieved outstanding detection performance while maintaining a remarkably small model size, despite having the longest training time of them all. Its precision and recall were also competitive, suggesting strong suitability for real-time underwater detection tasks where model compactness and inference speed are critical.

Ultimately, model selection depends on application-specific priorities. For highest classification accuracy and robustness, the Swin Transformer is preferred. For lightweight deployment without substantial loss in precision, EfficientNet offers an optimal compromise. For real-time detection and localization, YOLOv8 remains a highly effective solution.

These findings reinforce the importance of aligning model architecture choices with task-specific requirements, especially in challenging domains such as underwater object detection and species classification.

The insights gained from the comparative evaluation of YOLOv8, EfficientNet, and Swin Transformer models provide a foundation for drawing final conclusions regarding the effectiveness of modern deep learning architectures in underwater object detection and classification. The following chapter synthesizes these findings, discusses the limitations encountered, and outlines potential directions for future research.

5 Conclusion and Future Work

This chapter synthesizes the key findings obtained throughout the research project, highlighting the performance of the selected deep learning models on underwater fish species detection and classification tasks. Limitations encountered during the study are discussed to provide a critical perspective on the results. Finally, recommendations for future work are presented, aiming to guide further advancements in underwater object detection and classification using deep learning techniques.

5.1 Summary of Key Findings

This study assessed the performance of three deep learning architectures — YOLOv8, EfficientNet-B0, and the Swin Transformer — using the Fish4Knowledge dataset, with the objective of addressing the challenges inherent in underwater species detection and classification. Table C.5 summarizes the key results obtained from each model.

Among the evaluated models, the Swin Transformer achieved the highest overall classification performance, albeit at a bigger model size of ~ 323 MB, attaining a final test accuracy of 99.70%, a precision of 98.70%, and an F1-score of 97.60%. EfficientNet-B0 offered a strong balance between accuracy (99.09%) and computational efficiency, with a significantly smaller model footprint at ~ 50 MB compared to Swin Transformer. YOLOv8, optimized for real-time object detection, achieved a high mAP@0.5 of 0.982 while maintaining a compact model size of only ~ 12 MB.

These results demonstrate the viability of deep learning models for fine-grained underwater classification and detection tasks. They also highlight the trade-offs between model complexity, resource consumption, and predictive performance, offering important insights for real-world deployment scenarios.

5.2 Discussion of Limitations

While the results obtained in this study were promising, several limitations must be acknowledged.

First and foremost, the Fish4Knowledge dataset, although diverse, is limited in terms of ecological variability, primarily focusing on tropical reef environments near Taiwan. This restricts the models' ability to generalize to other underwater domains such as deep-sea, freshwater, or estuarine ecosystems, where lighting, turbidity, and species morphology differ significantly.

Secondly, the dataset exhibited inherent class imbalance, with certain fish species being heavily underrepresented. Although stratified sampling was employed during data splitting and macro-averaged metrics were used during evaluation to mitigate imbalance effects, the models may still be biased toward more frequent classes.

Thirdly, computational constraints restricted model scaling experiments. Larger variants of YOLOv8, EfficientNet, Swin Transformer (such as YOLOv8l, EfficientNet-B7 or Swin-B) were not explored due to GPU memory limitations and time constraint, potentially leaving room for even higher performance if larger architectures were deployed.

Finally, no extensive hyperparameter optimization (e.g., learning rate tuning, augmentation strategy optimization) was performed beyond the initial improvements. More aggressive hyperparameter tuning could further enhance model performance.

These limitations highlight areas where future studies could focus to build even more robust and generalizable underwater object detection and classification systems.

5.3 Recommendations for Future Work

Several directions can be pursued to extend and enhance this research, focusing on improving model generalization, robustness, adaptability, as well as real-world deployment capabilities.

The utmost important aspect is dataset expansion. Future work should involve training models on larger and ecologically diverse underwater datasets, incorporating environments like deep-sea, freshwater, estuarine habitats, and more. This broader environment coverage would help improve the

models' ability to generalize across varying underwater conditions.

Another promising direction is the use of synthetic data augmentation when training the models. Employing synthetic data generation frameworks to simulate a variety of underwater challenges — including turbidity, lighting distortion, and occlusion — could enhance the capability of trained models when deployed in unpredictable real-world environments.

Other than that, advanced domain adaptation techniques may offer a significant potential by incorporating unsupervised domain adaptation (UDA) and semi-supervised learning strategies. This would enable models to adapt to new underwater environments with minimal reliance on large amounts of labeled data, addressing one of the major limitations in underwater machine learning applications.

Further performance gains could be achieved through model scaling and optimization. Exploring larger backbone networks and systematically tuning hyperparameters, particularly within transformer-based architecture, could lead to substantial improvements in both classification and detection performance.

Last but not least, deployment-oriented research will be a crucial step for real-world application. Investigating model compression, pruning, and quantization techniques could make it feasible to deploy these models in real time on resource-constrained platforms, such as autonomous underwater vehicles (AUVs) and remotely operated vehicles (ROVs), expanding the practical utility of underwater species detection systems.

All in all, these future research directions could lead to significantly strengthen underwater detection models, not just by improving their technical performance but also by making them more practical for real-world use. By expanding both the reach and reliability of these systems, future work has the potential to make a real impact such as supporting marine research, aiding conservation efforts, and helping mankind better understand and protect aquatic ecosystems.

5.4 Final Closing Remarks

This study demonstrated the effectiveness of modern deep learning architectures for underwater fish species detection and classification, achieving strong results across YOLOv8, EfficientNet, and Swin Transformer models. Through careful dataset preparation, model selection, training optimization, and evaluation, the research provided valuable insights into the trade-offs between accuracy, computational efficiency, and generalizability.

While certain limitations were acknowledged, the findings pave the way for future advancements in developing more robust, efficient, and adaptable underwater detection systems. The integration of larger and more diverse datasets, coupled with emerging deep learning techniques, holds significant promise for expanding the capabilities of underwater object detection across a broader range of marine environments.

A Dataset Access

The original dataset used in this project is available at the following link [19] in the bibliography.

The preprocessed datasets used for training and evaluation in this project is available at the following Google Drive link:

[Click here](#) to access the dataset on Google Drive.

Please ensure you are logged into your University Google account to access the file if access restrictions are applied.

B Figures

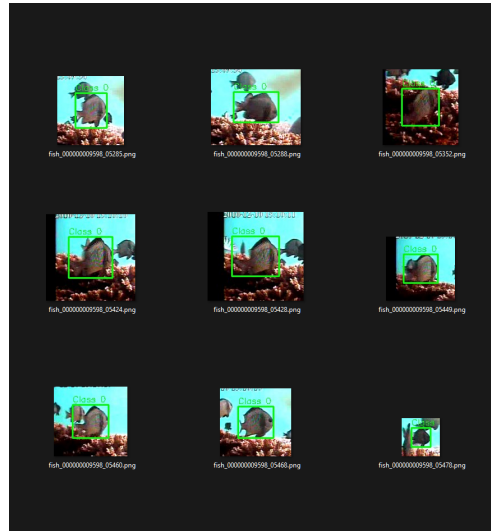


Figure B.1: Image sample of the train set with bounding box derived from the segmentation mask

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Haigal\Desktop\PRBDO & C:\Users\Haigal\anaconda3\envs\env_prbx\python.exe c:\Users\Haigal\Desktop\PRBDO\prepare_fish4k_yolo.py
Generating YOLO annotations from masks...
Class 01: 100% | 12112/12112 [00:31:00:00, 383.22it/s]
Class 02: 100% | 2683/2683 [00:06:00:00, 415.08it/s]
Class 03: 100% | 3593/3593 [00:09:00:00, 398.32it/s]
Class 04: 100% | 4849/4849 [00:11:00:00, 389.32it/s]
Class 05: 100% | 2534/2534 [00:06:00:00, 375.08it/s]
Class 06: 100% | 190/190 [00:00:00:00, 321.53it/s]
Class 07: 100% | 458/458 [00:00:00:00, 404.28it/s]
Class 08: 100% | 218/218 [00:00:00:00, 337.78it/s]
Class 09: 100% | 241/241 [00:00:00:00, 338.38it/s]
Class 10: 100% | 299/299 [00:00:00:00, 404.41it/s]
Class 11: 100% | 90/90 [00:00:00:00, 323.27it/s]
Class 12: 100% | 147/147 [00:00:00:00, 306.63it/s]
Class 13: 100% | 181/181 [00:00:00:00, 276.61it/s]
Class 14: 100% | 90/90 [00:00:00:00, 406.78it/s]
Class 15: 100% | 42/42 [00:00:00:00, 368.23it/s]
Class 16: 100% | 286/286 [00:00:00:00, 273.70it/s]
Class 17: 100% | 49/49 [00:00:00:00, 443.93it/s]
Class 18: 100% | 56/56 [00:00:00:00, 370.21it/s]
Class 19: 100% | 29/29 [00:00:00:00, 305.97it/s]
Class 20: 100% | 21/21 [00:00:00:00, 252.12it/s]
Class 21: 100% | 16/16 [00:00:00:00, 388.35it/s]
Class 22: 100% | 41/41 [00:00:00:00, 348.88it/s]
Class 23: 100% | 25/25 [00:00:00:00, 388.43it/s]

Splitting into train/val/test sets...
Train: 100% | 19159/19159 [04:15:00:00, 74.89it/s]
Val : 100% | 5474/5474 [01:07:00:00, 81.43it/s]
Test : 100% | 2737/2737 [00:33:00:00, 81.04it/s]

Generating sample images with bounding boxes for verification...
Preview: 100% | 20/20 [00:00:00:00, 65.70it/s]

All steps completed! Check 'fish4k-yolo/preview/' for bounding box verification.
PS C:\Users\Haigal\Desktop\PRBDO

```

Figure B.2: Screenshot of the terminal after the dataset preprocess

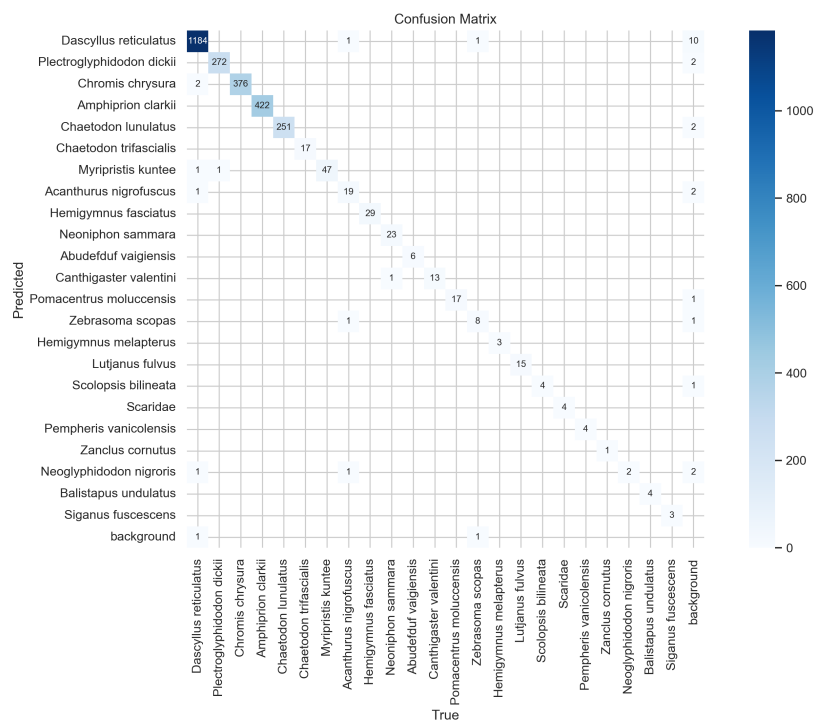


Figure B.3: YOLOv8 Confusion Matrix

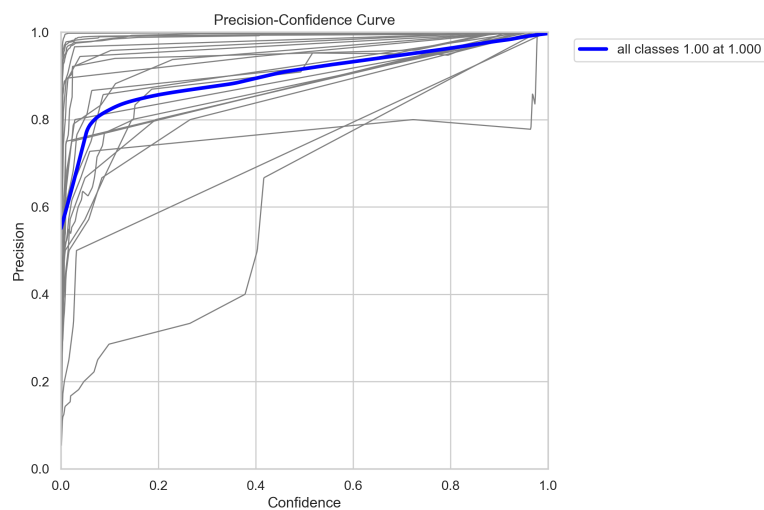


Figure B.4: YOLOv8 Precision-Confidence Curve

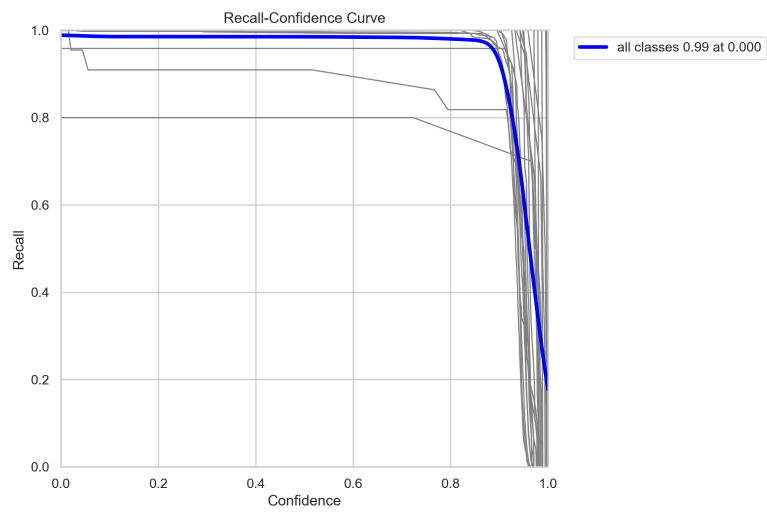


Figure B.5: YOLOv8 Recall-Confidence Curve

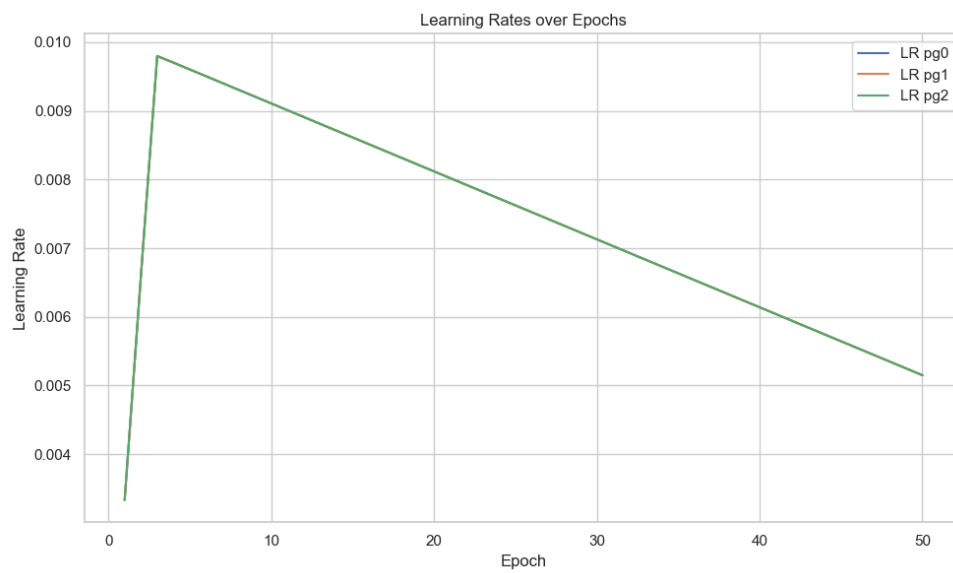


Figure B.6: YOLOv8 Learning Rates over Epochs

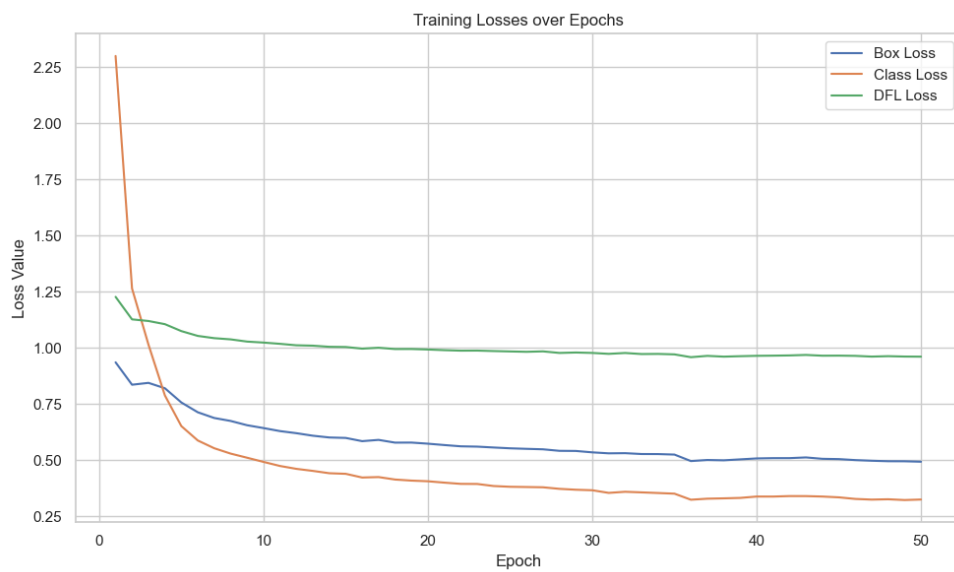


Figure B.7: YOLOv8 Trainig Losses over Epochs

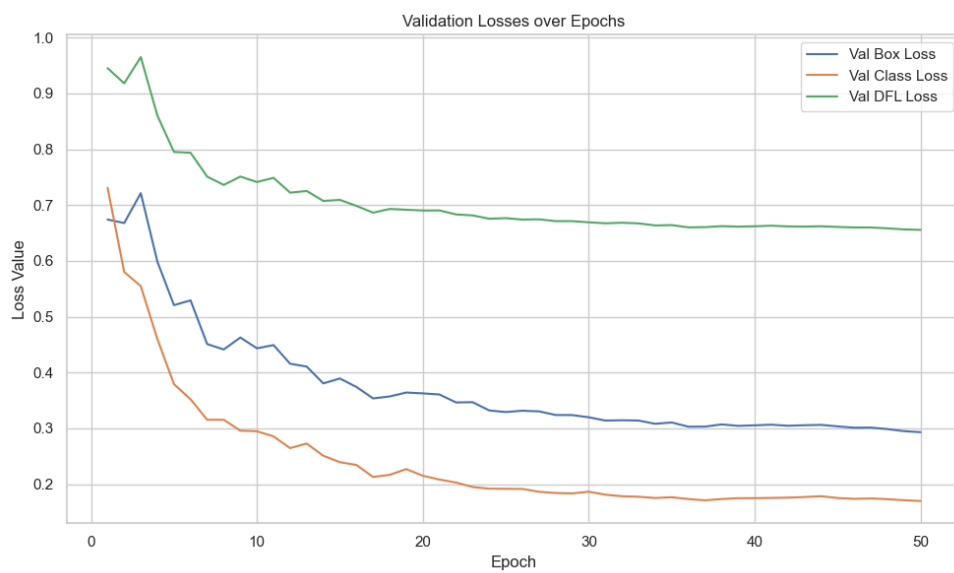


Figure B.8: YOLOv8 Validation Losses over Epochs

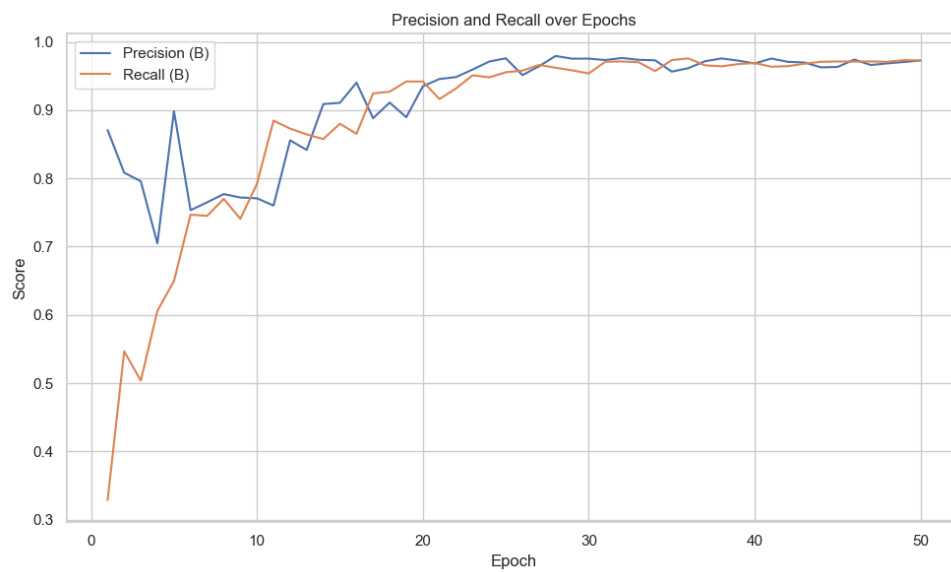


Figure B.9: YOLOv8 Precision and Recall over Epochs

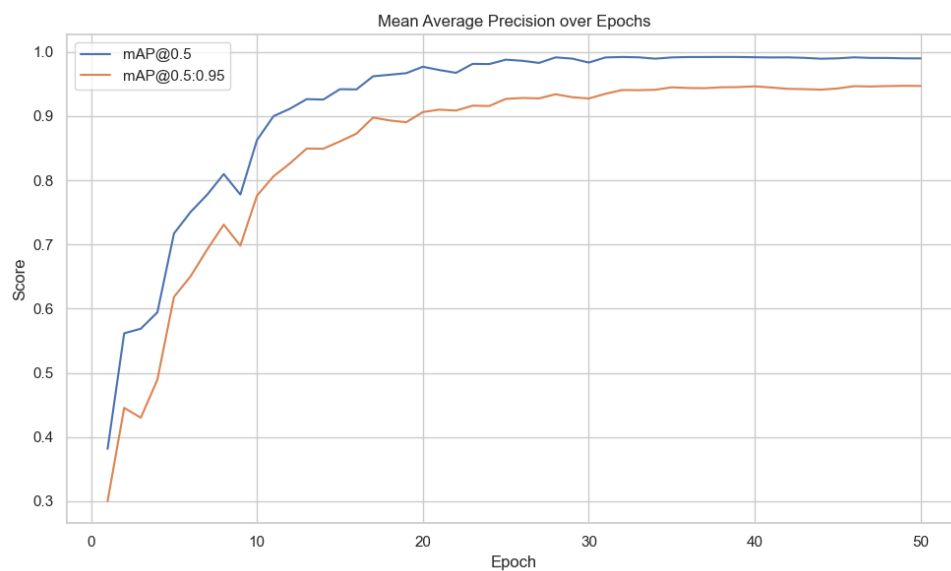


Figure B.10: YOLOv8 Mean Average Precision (mAP) over Epochs



Figure B.11: Sample of YOLOv8 Prediction on Test Set



Figure B.12: Sample of Ground Truth Annotation on Test Set

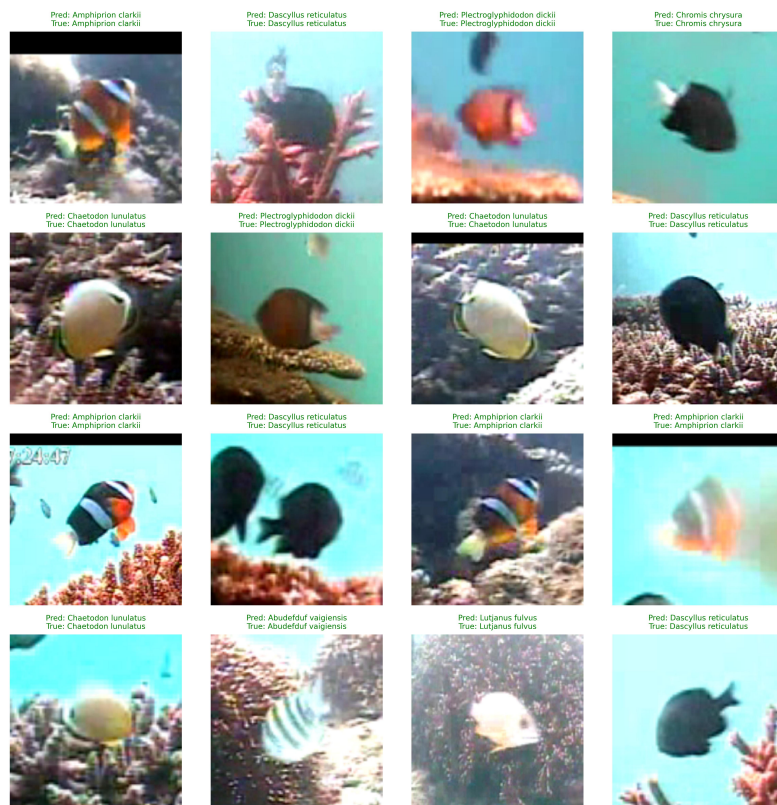


Figure B.13: Sample of EfficientNet Predictions on Test Set (Grid 1)

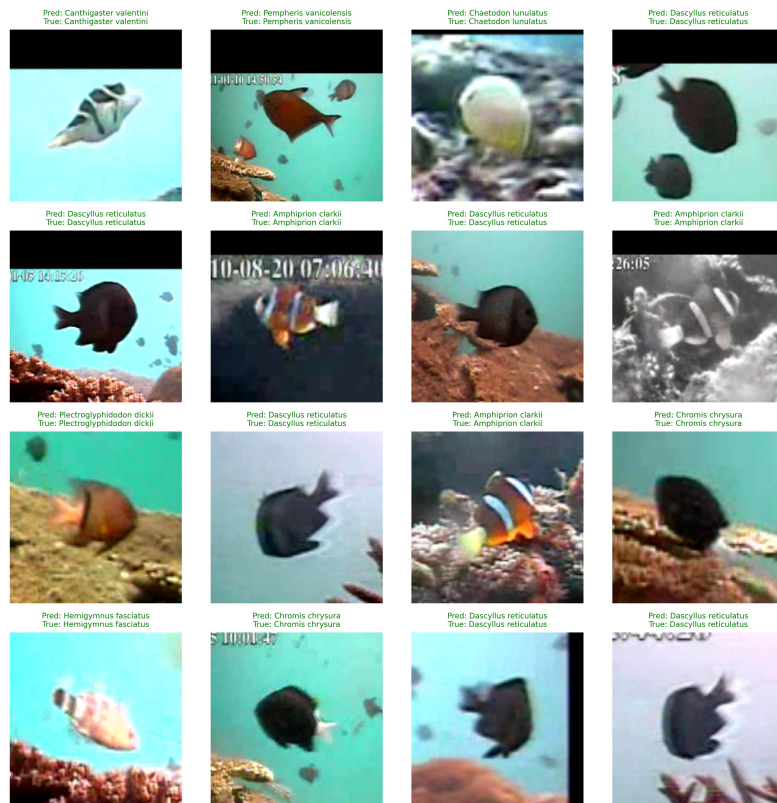


Figure B.14: Sample of EfficientNet Predictions on Test Set (Grid 2)

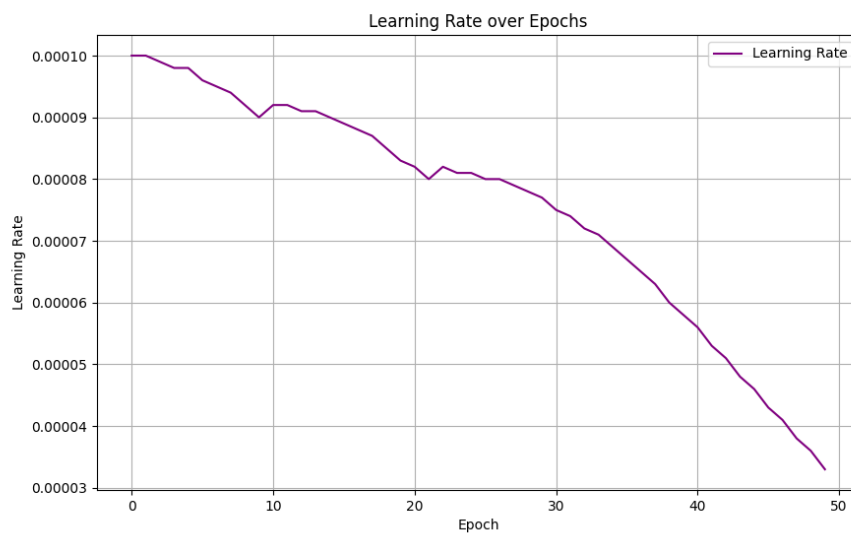


Figure B.15: Learning Rate Evolution over Epochs (Cosine Annealing)

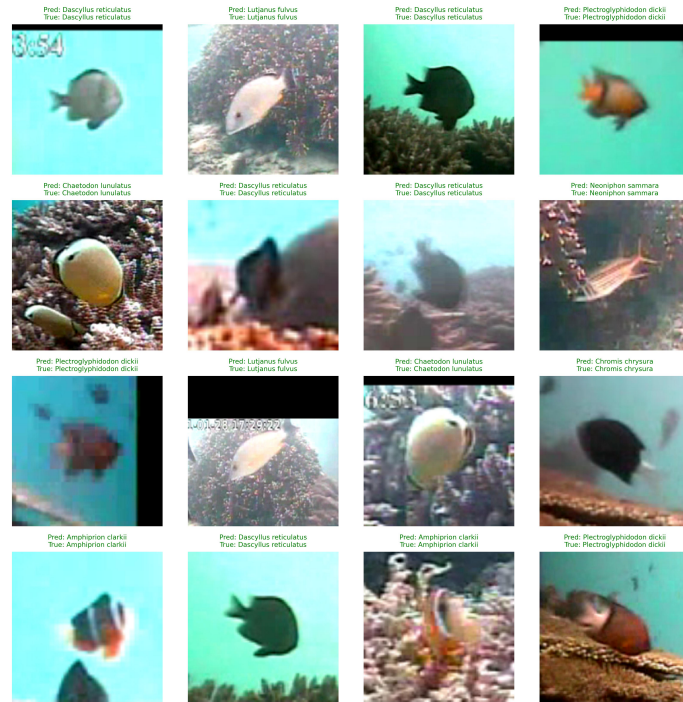


Figure B.16: Sample of Swin Transformer Predictions on Test Set (Grid 1)

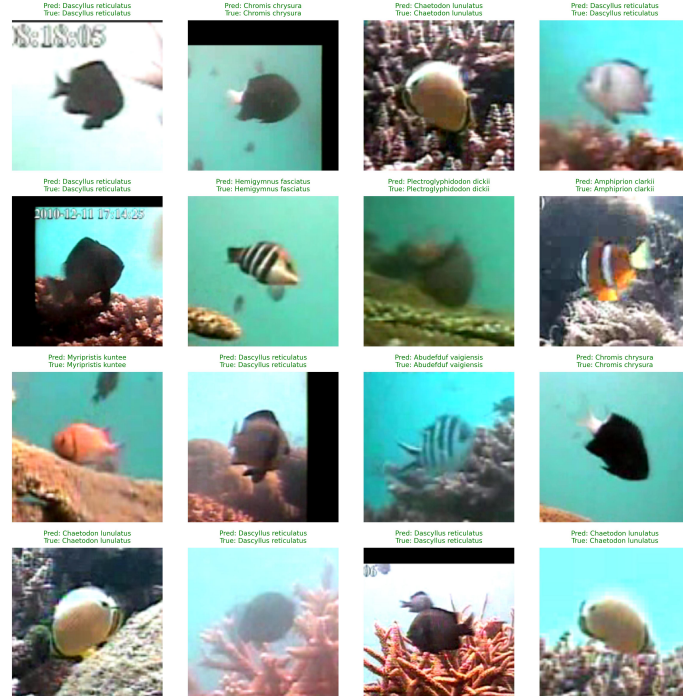


Figure B.17: Sample of Swin Transformer Predictions on Test Set (Grid 2)

C Tables

Table C.1: Choosing the Right Model: Trade-Offs and Application-Specific Considerations

Model	Dataset	Accuracy	Speed	Small Object Detection	Suitability for Real-Time Applications
YOLOv8	DUO, TrashCan	High	Very High	Moderate	Ideal for real-time tracking [6]
CEH-YOLO	DUO, UT-DAC2020	Very High	Very High	Excellent	Best for detecting small/camouflaged objects [1]
Faster R-CNN	URPC2020	Very High	Low	Excellent	Not suitable for real-time use [6]
Swin Transformer	ADE20K, COCO	Very High	Moderate	Excellent	Promising, but not fully optimized for speed [12]
EfficientNet	COCO	High	High	Moderate	Suitable for edge devices with moderate speed-accuracy trade-off [6]
Hybrid CNN + Transformer (FBDPN)	URPC2020, Aquarium	Very High	Moderate	Excellent	Balanced precision and efficiency for practical applications [14]

Table C.2: Summary of Datasets Considered for Fish Detection and Classification

Dataset	Main Focus	Advantages	Limitations
DeepFish [15]	Segmentation, Classification, Localization	Real underwater imagery; structured masks available	Partial species-specific labels; too many generic "fish" class
Large-Scale Fish Dataset [16]	Segmentation, Classification	High-quality masks; 9 seafood classes; organized data	Captured in lab settings; limited species diversity; no bounding boxes
NOAA Labeled Fishes in the Wild [17]	Detection	Bounding box annotations; real underwater conditions	Non-standard annotation formats (.vec/.dat); limited test set usability
WildFish Dataset [18]	Detection, Classification	Massive dataset (200k+ images, 1,960 species); species-level labels	Access restrictions; enormous size (20TB); impractical for current scope
Fish4Knowledge [19]	Detection, Tracking, Classification	Species diversity; paired masks; underwater realism; public accessibility	Some class imbalance; masks require conversion to bounding boxes

Table C.3: Swin Transformer Early Training Results Before Improvements

Epoch	Train Loss	Train Accuracy	Validation Loss	Validation Accuracy
0	~1.618	~48.2%	~1.383	~53.0%
1	~1.580	~48.3%	~1.549	~50.6%
2	~1.696	~46.2%	~1.688	~43.8%
3	~1.796	~44.4%	~1.807	~44.3%
4	~1.806	~44.3%	~1.802	~44.3%
5	~1.803	~44.3%	~1.797	~44.3%

Table C.4: Swin Transformer Training Results After Improvements

Epoch	Train Loss	Train Accuracy	Validation Loss	Validation Accuracy
0	~0.261	~92.7%	~0.052	~98.6%
1	~0.137	~95.8%	~0.012	~99.6%
2	~0.113	~96.5%	~0.013	~99.6%
3	~0.099	~97.1%	~0.016	~99.5%
4	~0.098	~96.9%	~0.013	~99.5%

Table C.5: Comparative Performance Metrics Across Models Trained on Fish4Knowledge Dataset

Model	Precision	Recall	F1-Score	Final Test Accuracy / mAP	Model Size	Train Time (50 Epochs)
YOLOv8n (Detection)	0.973	0.978	0.970	mAP@0.5: 0.982, mAP@0.5:0.95: 0.933	~12 MB	~3 hours
EfficientNet-B0 (Classification)	0.985	0.948	0.962	0.991	~50 MB	~2 hours
Swin Transformer Tiny (Classification)	0.987	0.974	0.976	0.997	~323 MB	~2.5 hours

Bibliography

- [1] J. Feng and T. Jin, 'Ceh-yolo: A composite enhanced yolo-based model for underwater object detection,' *Ecological Informatics*, vol. 82, p. 102 758, 2024. DOI: 10.1016/j.ecoinf.2024.102758.
- [2] S. Zheng, R. Wang and L. Wang, 'Underwater fish object detection with degraded prior knowledge,' *Electronics*, vol. 13, no. 12, p. 2346, 2024. DOI: 10.3390/electronics13122346.
- [3] H.-G. Kim, J. Seo and S. M. Kim, 'Underwater optical-sonar image fusion systems,' *Sensors*, vol. 22, no. 21, p. 8445, 2022. DOI: 10.3390/s22218445.
- [4] X. Ji, G.-P. Liu and C.-T. Cai, 'Collaborative framework for underwater object detection via joint image enhancement and super-resolution,' *Journal of Marine Science and Engineering*, vol. 11, no. 9, p. 1733, 2023. DOI: 10.3390/jmse11091733.
- [5] M. Hamzaoui, M. Ould-Elhassen Aoueilleyine, L. Romdhani and R. Bouallegue, 'An improved deep learning model for underwater species recognition in aquaculture,' *Fishes*, vol. 8, no. 10, p. 514, 2023. DOI: 10.3390/fishes8100514.
- [6] W. Zariful, N. Tashim, T. Lim *et al.*, 'Analysis of deep learning algorithms for prawn aquaculture in a challenging environment,' in *2023 6th International Conference on Applied Computational Intelligence in Information Systems (ACIIS)*, IEEE, 2023, pp. 1–5. DOI: 10.1109/ACIIS59385.2023.10367401.
- [7] S. Zhu, X. Xia, Q. Zhang and K. Belloulata, 'An image segmentation algorithm in image processing based on threshold segmentation,' in *Third International IEEE Conference on Signal-Image Technologies and Internet-Based System*, IEEE, 2008, pp. 673–679. DOI: 10.1109/SITIS.2007.116.
- [8] C. Fu, W. Yuan, J. Xiao, R. Liu and X. Fan, 'Joint perceptual learning for enhancement and object detection in underwater scenarios,' *IEEE Transactions on Image Processing*, 2023, Preprint available on arXiv:2307.03536. [Online]. Available: <https://arxiv.org/abs/2307.03536>.

- [9] L. Folkman, K. A. Pitt and B. Stantic, 'A data-centric framework for combating domain shift in underwater object detection with image enhancement,' *Applied Intelligence*, vol. 55, no. 272, 2025. DOI: 10.1007/s10489-024-06224-0.
- [10] D. Warfield, *Yolo: Intuitively and exhaustively explained*, Accessed: 2025-04-22, 2023. [Online]. Available: <https://iaee.substack.com/p/yolo-intuitively-and-exhaustively>.
- [11] Z. Deng, *The architecture of faster r-cnn*, Accessed: 2025-04-22, 2018. [Online]. Available: https://www.researchgate.net/figure/The-architecture-of-Faster-R-CNN_fig2_324903264.
- [12] Z. Liu, Y. Lin, Y. Cao *et al.*, 'Swin transformer: Hierarchical vision transformer using shifted windows,' in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 10 012–10 022. DOI: 10.1109/ICCV48922.2021.00989.
- [13] M. Tan and Q. V. Le, 'Efficientnet: Rethinking model scaling for convolutional neural networks,' in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, vol. 97, PMLR, 2019, pp. 6105–6114. [Online]. Available: <http://proceedings.mlr.press/v97/tan19a.html>.
- [14] X. Ji, S. Chen, L.-Y. Hao, J. Zhou and L. Chen, 'Fbdpn: Cnn-transformer hybrid feature boosting and differential pyramid network for underwater object detection,' *Expert Systems with Applications*, vol. 256, p. 124 978, 2024. DOI: 10.1016/j.eswa.2024.124978.
- [15] K. Alzayat, *Deepfish: A dataset for fish detection and classification in underwater environments*, 2020. [Online]. Available: <https://github.com/alzayats/DeepFish>.
- [16] W. Crow, *A large scale fish dataset*, 2021. [Online]. Available: <https://www.kaggle.com/datasets/crowww/a-large-scale-fish-dataset>.
- [17] N. Fisheries, *Labeled fishes in the wild*, 2020. [Online]. Available: <https://www.fisheries.noaa.gov/west-coast/science-data/labeled-fishes-wild>.
- [18] P. Zhuang, *Wildfish: A dataset for underwater fish detection and classification*, 2022. [Online]. Available: <https://github.com/PeiqinZhuang/WildFish>.
- [19] R. B. Fisher and colleagues, *Fish4knowledge ground truth dataset for species recognition*, 2012. [Online]. Available: <https://homepages.inf.ed.ac.uk/rbf/Fish4Knowledge/GROUNDTRUTH/RECOG/>.
- [20] Ultralytics, *Yolov8 architecture explained*, 2023. [Online]. Available: <https://yolov8.org/yolov8-architecture/>.

- [21] A. Sarkar, *Understanding efficientnet: The most powerful cnn architecture*, Medium Article, 2020. [Online]. Available: <https://arjun-sarkar786.medium.com/understanding-efficientnet-the-most-powerful-cnn-architecture-eaeb40386fad>.
- [22] T. Chau, *Explanation: Swin transformer*, Medium Article, 2022. [Online]. Available: <https://chautuankien.medium.com/explanation-swin-transformer-93e7a3140877>.
- [23] G. Jocher, A. Chaurasia and J. Qiu, *Ultralytics yolov8*, version 8.0.0, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [24] Ultralytics, *Yolov8 detection tasks documentation*, Online, <https://docs.ultralytics.com/tasks/detect/>, 2023.
- [25] Z. Zheng, P. Wang, W. Liu, J. Li, L. Ye and D. Ren, 'Distance-iou loss: Faster and better learning for bounding box regression,' *arXiv preprint arXiv:1911.08287*, 2020.
- [26] C.-Y. Zhang, B. Cheng, Y. Wei *et al.*, 'Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection,' *arXiv preprint arXiv:2006.04388*, 2021.
- [27] GeeksforGeeks, *Efficientnet architecture*, 2024. [Online]. Available: <https://www.geeksforgeeks.org/efficientnet-architecture/>.
- [28] I. Loshchilov and F. Hutter, 'Decoupled weight decay regularization,' in *7th International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://arxiv.org/abs/1711.05101>.
- [29] Z. Liu, Y. Lin, Y. Cao *et al.*, 'Swin transformer: Hierarchical vision transformer using shifted windows,' *CoRR*, vol. abs/2103.14030, 2021. arXiv: 2103.14030. [Online]. Available: <https://arxiv.org/abs/2103.14030>.
- [30] N. V. Otten, *Cosine annealing in machine learning simplified: Understand how it works*, 2024. [Online]. Available: <https://spotintelligence.com/2024/04/29/cosine-annealing-in-machine-learning/>.
- [31] B. Bodner, *10 must-have pytorch schedulers you didn't know you need*, 2024. [Online]. Available: <https://medium.com/@benjybo7/10-must-have-schedulers-that-will-boost-your-models-performance-2ff0c446ac98>.